TRAILLINK: HYBRID COMMUNICATION & SAFETY SYSTEM

INDIVIDUAL THESIS

Thanujana D O

(IT22148704)

Individual component: Communication Mesh Intelligence & Compass

Group ID: R26-IT-135

Report submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) Degree in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

I declare that this is my own work and this thesis does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my thesis, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Thanujana D O

Student ID: IT22148704

Signature:  ____ Date: 10/09/2026 _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr.Vishan Jayasinghearachchi		
Co-supervisor: Ms.Sashini Hathurusinghe		

ABSTRACT

Reliable communication among outdoor field teams is critically challenged by the absence of cellular infrastructure in remote environments such as hiking trails, camping grounds, and disaster-affected areas. This thesis presents the design, implementation, and evaluation of the Connectivity, Mesh Intelligence, and Compass component of TrailLink, a hybrid mobile communication application for Android. The component addresses the problem of providing situational connectivity awareness and bounded packet forwarding on ordinary smartphones without reliance on any external network infrastructure. Four integrated subsystems are designed and implemented: (1) a backend reachability detector that distinguishes confirmed server access from mere network-interface presence and drives a hysteresis-based connection-mode state machine; (2) a peer discovery and session management layer built on the Google Nearby Connections SDK using the P2P_CLUSTER strategy with channel-scoped peer validation; (3) a mesh intelligence pipeline that records observable link events: Received Signal Strength Indicator (RSSI), acknowledgement round-trip time (ACK RTT), packet success rate, retry count, and disconnect count; it computes a bounded composite quality score (0–100), classifies it into five labelled tiers (Excellent, Good, Fair, Weak, Lost), detects improving or degrading trends through split-half score comparison, and generates plain-language user guidance; and (4) a Network Compass feature that measures HTTP download throughput against the TrailLink backend, geotags each sample on-device using GPS, and provides geodesic directional guidance toward the best recorded location. A controlled relay mechanism forwards eligible safety and control packet types through connected peers using TTL budgets, hop-count limits, channel scoping, and origin exclusion to prevent uncontrolled circulation. Functional evaluation demonstrates correct quality label classification across all five tiers, accurate trend detection across 12 synthetic sequences, and successful relay forwarding and rejection enforcement in two-device configurations. The component contributes an observable, deterministic, and privacy-preserving approach to connectivity intelligence on commodity hardware operating in infrastructure-free settings.

Keywords: offline communication, mesh networking, peer-to-peer, link quality estimation, Network Compass

ACKNOWLEDGEMENTS

The completion of this thesis would not have been possible without the support and guidance of several individuals, to whom I am sincerely grateful.

I would like to express my deepest gratitude to my thesis supervisor, Mr. Vishan Jayasinghearachchi, whose expert guidance, constructive feedback, and unwavering patience were instrumental throughout every stage of this research. Their direction in shaping the design of the Connectivity, Mesh Intelligence, and Compass component was invaluable in maintaining both academic rigour and practical relevance.

I am equally grateful to the academic staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for their support and expertise in mobile computing and software architecture, which directly influenced the methodology adopted in this work.

I would also like to acknowledge the contribution of my fellow research team members who developed the complementary components of the TrailLink platform. Their collaborative spirit and willingness to align technical interfaces across components made the end-to-end system integration possible.

This research was conducted independently without external funding. I acknowledge Google LLC for providing the Nearby Connections API and the Flutter framework, which were central to the implementation of this component.

Finally, I am deeply thankful to my family for their enduring support, patience, and encouragement throughout the duration of this research.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
Abstract ii
ACKNOWLEDGEMENTS iv
TABLE OF CONTENTS v
LIST OF FIGURES viii
LIST OF TABLES ix
LIST OF ABBREVIATIONS x
LIST OF APPENDICES xii
1 INTRODUCTION 1
1.1 Overall Project Context 2
1.2 Individual Component Context 3
1.3 Problem Statement 5
1.4 Research Gap 6
1.5 Aim and Objectives 7
1.6 Research Questions or Hypotheses 8
1.7 Scope Limitations and Assumptions 9
1.8 Individual Contribution 10
1.9 Thesis Structure 11
2 LITERATURE REVIEW AND RELATED WORK 12
2.1 Review Method and Scope 12
2.2 Domain and Theoretical Foundations 12
2.3 Related Research 14
2.4 Existing Systems Products and Techniques 16
2.5 Gap Synthesis and Conceptual Framework 17
3 RESEARCH METHODOLOGY 19
3.1 Research Design 19
3.2 Data Participants or Study Materials 20
3.3 Experimental or Study Procedure 21
3.4 Measures and Evaluation Metrics 22
3.5 Data Analysis 23
3.6 Ethics Privacy Security and Responsible AI 24
3.7 Reproducibility and Research Artefacts 25
4 PRODUCT AND SOLUTION DESIGN 26
4.1 Requirements 26

4.2 Component Architecture Within the Complete Solution 27
4.3 Design Alternatives and Decisions 29
4.4 Detailed Design 31
4.4.1 Backend reachability and connection mode 31
4.4.2 Peer discovery and session management 33
4.4.3 Mesh intelligence pipeline 34
4.4.4 Network Compass 37
4.4.5 Relay service 39
4.5 Standards and Professional Practice 40
5 IMPLEMENTATION INTEGRATION AND TESTING 41
5.1 Development Environment and Technology Stack 41
5.2 Component Implementation 42
5.3 Integration with the Other Components 44
5.4 Verification and Software Testing 46
5.4.1 Unit testing 46
5.4.2 Integration testing 47
5.4.3 Guidance engine testing 48
5.5 Deployment and Operation 48
6 RESULTS AND VALIDATION 51
6.1 Research Results 51
6.2 Product and Technical Results 52
6.3 User or Stakeholder Validation 53
6.4 Integration Results 54
6.5 Summary of Findings 55
7 DISCUSSION 57
7.1 Interpretation of Findings 57
7.2 Comparison with Literature Baselines and Existing Solutions 58
7.3 Research Contribution 59
7.4 Product Contribution 60
7.5 Limitations and Threats to Validity 60
7.6 Implications 61
8 COMMERCIALISATION AND SUSTAINABILITY 63
8.1 Value Proposition and Target Users 63
8.2 Market and Competitive Position 63
8.3 Business or Sustainability Model 64
8.4 Scalability Intellectual Property and Compliance 65
8.5 Contribution to the Sustainable Development Goals 65

9 CONCLUSION AND FUTURE DIRECTIONS 67
9.1 Conclusion 67
9.2 Achievement of Objectives 68
9.3 Individual Research and Product Contributions 68
9.4 Future Directions 69
REFERENCES 71
GLOSSARY 72
APPENDICES 73
Appendix A: Research Instruments or Extended Results 73
Appendix B: Academic Integrity and AI Use Disclosure 74

LIST OF FIGURES

Figure No.	Figure Title	Page
Figure 2.1	Conceptual framework for Connectivity Intelligence – from hardware events to user guidance.	18
Figure 4.1	TrailLink component architecture showing connectivity layer and inter-component interfaces.	28
Figure 4.2	ConnectionModeController state machine (Online / Offline / Reconnecting / Unstable)	33
Figure 4.3	Peer discovery and session lifecycle sequence diagram.	34
Figure 4.4	Mesh intelligence pipeline data flow (event → sample → score → trend → guidance → UI)	34
Figure 5.1	Integration data flow for an offline SOS event across all four components.	45
Figure 5.2	ConnectivityGuidanceScreen – speedometer, best internet spot card, and peer quality ranking	50

LIST OF TABLES

Table No.	Table Title	Page
Table 2.1	Comparison of Related Offline Communication Systems	16
Table 4.1	Component Requirements	26
Table 4.2	PeerQualityCalculator Scoring Rules	35
Table 5.1	Development Environment and Technology Stack	41
Table 5.2	Source File Map – Connectivity, Mesh Intelligence, and Compass Component	42
Table 6.1	Research Questions and Evidence Summary	52
Table 6.2	Summary of Product-Level Test Results	52
Table A.1	Extended PeerQualityCalculator Scoring Examples	73
Table A.2	GuidanceEngine Test Cases	73

LIST OF ABBREVIATIONS

Abbreviation	Description
ACK	Acknowledgement
ADB	Android Debug Bridge
AI	Artificial Intelligence
AODV	Ad hoc On-Demand Distance Vector
API	Application Programming Interface
APK	Android Package
BLE	Bluetooth Low Energy
DSR	Design Science Research
DTN	Delay-Tolerant Networking
ETX	Expected Transmission Count
GDPR	General Data Protection Regulation
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISM	Industrial, Scientific and Medical
LQE	Link Quality Estimation
LTE	Long-Term Evolution

MANET	Mobile Ad Hoc Network
MDM	Mobile Device Management
ML	Machine Learning
NGO	Non-Governmental Organisation
OLSR	Optimized Link State Routing
P2P	Peer-to-Peer
PRR	Packet Reception Rate
PTT	Push-to-Talk
RFC	Request for Comments
RSSI	Received Signal Strength Indicator
RTT	Round-Trip Time
SAR	Search and Rescue
SDG	Sustainable Development Goal
SDK	Software Development Kit
SNR	Signal-to-Noise Ratio
TTL	Time To Live
UI	User Interface
USB	Universal Serial Bus

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Research Instruments or Extended Results	73
Appendix B	Academic Integrity and AI Use Disclosure	74

1 INTRODUCTION

Hikers, campers and field teams often work where there is no mobile coverage, and a phone that shows a Wi-Fi or mobile icon still may not be able to reach any server. The phone knows which network it is joined to, but not whether messages sent through it will arrive. Once teammates switch to phone-to-phone links, a second problem appears: the app has no way to say which nearby phone has a strong link, which one is fading, or where to walk to get better coverage. That information is worth more than most apps use. Read together, signals such as round-trip times, packet results and disconnect events show which links are healthy, which are about to fail, and how a team's connection changes as it moves [2], [8], [9]. In this thesis I describe how I used those signals to build connectivity intelligence inside TrailLink: Hybrid Communication & Safety System for outdoor field teams of two to six people.

The Connectivity, Mesh Intelligence and Compass component is my individual contribution to TrailLink. Its job is to tell the rest of the application, and the user, what the connection is really like before a message or an SOS is sent. It sits at the foundation of the application, so the other three components depend on it. The Communication Core uses its connection mode to choose a send path, the Voice and Push-to-Talk component checks it before starting Live Radio, and the Emergency SOS component reads it to decide between the backend and the offline relay. The component runs inside the real Android application and is not a stand-alone prototype. Team members see its output on the ConnectivityGuidanceScreen, which shows a plain-language guidance message, a ranked list of connected peers, and a compass pointing towards the best internet spot found so far.

The component is built from five cooperating parts. Backend reachability detection sends an HTTP HEAD request to the backend every 15 seconds and applies simple hysteresis rules to place the device in one of four modes: Online, Offline, Reconnecting or Unstable [4]. Peer discovery and session management use the Google Nearby Connections API with the P2P_CLUSTER strategy, and only peers on the same channel are accepted [2]. The mesh intelligence pipeline turns raw events into a composite quality score from 0 to 100, labelled Excellent, Good, Fair, Weak or Lost,

adds a trend (Improving, Stable or Degrading), and passes the result to a rule-based guidance engine [8], [9]. Network Compass measures ping and download speed against the backend, stores each measurement with a GPS fix on the device only, and points the user to the best location measured so far [7]. Finally, a relay service forwards only safety and control packets, within a time-to-live (TTL) limit, a maximum hop count of 5 and the active channel, so that the number of copies stays bounded [6], [12]. I chose rule-based scoring over machine learning because the rules can be inspected and explained, and because on-device inference would drain the battery in the field.

1.1 Overall Project Context

TrailLink is a Hybrid Communication & Safety System for Android designed to support hikers, campers, and field teams operating in environments where cellular or Wi-Fi infrastructure is absent or unreliable [1]. The application supports two primary operating modes: an Online Trip mode, which uses a Node.js and Express backend, a MongoDB database, and Socket.IO for cloud-based group communication; and an Offline Trip mode, which relies exclusively on direct phone-to-phone communication via the Google Nearby Connections API without any network infrastructure [2].

The project was undertaken as a four-component group research effort. Each component represents a distinct functional domain: (1) Connectivity and Mesh Intelligence, (2) Communication Core, (3) Voice and PTT Communication, and (4) Location, Emergency SOS, Mapping and Syncing. Although the application is deployed as a single Flutter mobile application, each component has defined boundaries and interfaces to the others [1]. The mobile frontend is built with Flutter and Dart [3] using Riverpod for reactive state management and GoRouter for declarative navigation, while local persistence is achieved through SQLite via the sqflite package.

Outdoor team communication is a safety-critical concern in both recreational and professional field contexts. In environments such as forested mountain terrain, deep valleys, and coastal wilderness areas, cellular coverage is absent over large swathes frequented by hikers and professional field teams alike. Satellite communication devices such as the Garmin inReach provide coverage in these environments but

require additional hardware investment, subscription fees, and extra equipment weight that many users are unwilling to accept. TrailLink addresses this gap by transforming consumer smartphones that field team members already carry into a self-contained communication network, requiring no additional hardware beyond a standard Android device.

Building a reliable offline communication system on consumer smartphones introduces several distinct technical challenges. The radio hardware in consumer devices (Bluetooth and Wi-Fi) was not designed for the range or reliability required of professional field communication hardware. Bluetooth Classic and BLE operate at 2.4 GHz with effective ranges of 10–100 metres in free space, substantially reduced by foliage, terrain obstacles, and interference from other devices [11]. Consumer Android firmware implementations vary significantly in how they expose radio signal metrics to applications: some devices provide RSSI values through the Nearby Connections API, while others expose no signal quality information whatsoever, requiring applications to infer link quality from application-layer observations. Battery life is a further constraint: continuous Bluetooth advertising and discovery scanning consume power, and outdoor teams rely on their smartphones for navigation and emergency alerting in addition to communication. These constraints define the engineering environment within which the Connectivity, Mesh Intelligence, and Compass component was designed.

1.2 Individual Component Context

This thesis documents the individual research and development contribution for the "Connectivity, Mesh Intelligence and Compass" component. This component is the foundational connectivity layer of the TrailLink system: it determines whether the application can reach its backend server, establishes and manages nearby phone-to-phone links using the Google Nearby Connections SDK [2], estimates the quality of those links from observable application-layer evidence, guides users toward better connectivity, and implements bounded application-layer packet forwarding across the mesh of connected devices.

The component encompasses four sub-functions that together form a coherent connectivity intelligence system: backend reachability detection, peer discovery and session management, mesh intelligence (quality scoring, trend analysis, and guidance generation), and the Network Compass feature. It provides foundational services to all other components: the Communication Core depends on connection state and packet transport; Voice and PTT uses Nearby links for radio eligibility decisions; Location and Emergency SOS relies on this component's transport paths for safety-critical packet forwarding.

The foundational nature of this component means that failures within it propagate immediately to all other components. If the connection mode state machine incorrectly classifies the device as Online when it is actually offline, the Communication Core will attempt to send messages through the backend and fail silently, with messages lost rather than queued for offline delivery. If peer discovery fails to correctly scope connections to the active trip channel, team members from different trips could appear in each other's peer lists, causing privacy violations and operational confusion. If relay forwarding propagates packets without TTL constraints, a single SOS message could circulate indefinitely, consuming bandwidth and battery on all connected devices. The design of this component therefore prioritises correctness and safety above all other concerns.

The overarching design philosophy of the component is observable, deterministic, and explainable intelligence. All quality scoring, trend detection, guidance generation, and relay forwarding decisions are made by rule-based algorithms whose behaviour can be traced from inputs to outputs without ambiguity. This transparency is essential in safety-critical applications where users must trust the guidance they receive and where developers must audit the relay forwarding behaviour of the system. The use of machine learning or probabilistic models was explicitly rejected during the design phase in favour of deterministic rules that produce the same output for the same inputs regardless of training data, model state, or inference context [15].

1.3 Problem Statement

Field teams operating in infrastructure-free outdoor environments face two compounding communication challenges. First, the absence of cellular coverage eliminates conventional messaging and renders users unable to determine whether their device is genuinely isolated from a backend service or merely on a network interface without active backend reachability [4]. Mobile operating systems routinely report a Wi-Fi association even when no application-level server is accessible, creating misleading connectivity indicators that result in incorrect application mode decisions and silent message loss.

Second, even when direct phone-to-phone Bluetooth or Wi-Fi links are established through the Nearby Connections SDK, users lack real-time understanding of link quality, degradation trends, and the likelihood that a safety-critical message will be successfully relayed to a distant teammate through intermediate devices [5]. Finally, when a user seeks internet access in a partial-coverage area, no integrated tool guides them spatially toward previously measured high-quality signal spots while preserving their location privacy.

The first challenge is illustrated by the following concrete scenario. A hiking team of four arrives at a mountain hut that offers Wi-Fi. Each team member's phone successfully associates with the hut's Wi-Fi network, and Android's status bar displays a full Wi-Fi signal icon. However, the hut's satellite internet uplink is offline due to adverse weather. Android's ConnectivityManager reports the network as connected because it detects a valid local-area network association. Without backend reachability checking, TrailLink would attempt to send all messages through the backend socket connection, displaying "Sending..." indefinitely with no indication that messages are not reaching teammates. The Connectivity component resolves this by performing an explicit HTTP HEAD request to the backend health endpoint, detecting the uplink failure within the first 15-second check cycle, and transitioning the application to Offline mode so that all subsequent messages are routed through the Nearby Connections channel [4].

The second challenge is illustrated by the following scenario. Two team members, Alice and Bob, are connected via Bluetooth while hiking. Bob is stationary at a viewpoint while Alice continues. As Alice moves further away and behind a rocky outcrop, her connection to Bob degrades: RSSI drops below −80 dBm, ACK round-trip times increase from 150 ms to over 1,200 ms, and packet delivery failures begin occurring. Without link quality intelligence, neither Alice nor Bob receives any warning until messages actually start failing. With the mesh intelligence pipeline active, Bob's ConnectivityGuidanceScreen transitions from showing Alice's quality as Good to Fair to Weak over successive measurement cycles, and the GuidanceEngine generates the message "Signal to Alice is getting weaker. Move back toward your previous position." This provides actionable spatial guidance before communication fails completely [5], [8].

1.4 Research Gap

A review of the literature and existing tools identifies the following specific gaps addressed by this component:

Gap 1: Conflation of network-interface presence with application-layer reachability [4]. Consumer devices report a Wi-Fi or mobile data association without confirming that any specific application backend is reachable. Most mobile P2P communication tools treat network-interface detection as a proxy for full connectivity, leading to incorrect mode decisions and failed transmissions. In safety-critical outdoor applications, a message silently lost because the application incorrectly assumed backend reachability could have serious consequences for team coordination.

Gap 2: Absence of observable link quality metrics in consumer-smartphone P2P communication tools [5]. Existing solutions such as Meshtastic and goTenna rely on dedicated hardware with built-in RSSI and SNR reporting. Consumer smartphone applications using Bluetooth or Wi-Fi Direct do not systematically surface link-quality estimates at the application layer, leaving users without actionable information about connection health or trajectory. This gap is

particularly consequential in outdoor environments where link quality varies rapidly with terrain and distance.

Gap 3: Lack of trip-scoped and channel-scoped relay controls [6]. Application-layer relay mechanisms on commodity hardware risk broadcast storms without rigorous forwarding constraints. A naive flooding approach in a team of 6 devices with a message TTL of 10 hops could theoretically generate up to 6^10 packet copies in the worst case, which is unacceptable for battery-constrained consumer devices. Existing systems either rely on the transport SDK for routing or implement naive flooding without TTL, hop-count, or channel-scoping constraints.

Gap 4: No integrated spatial guidance for internet signal optimisation on privacy-preserving consumer devices [7]. Mobile applications do not combine network speed probing with geolocation sampling to guide users spatially toward better signal coverage without uploading location data to a remote service. Existing signal-mapping solutions require users to share their GPS coordinates with a central server, which is unacceptable in privacy-sensitive outdoor scenarios.

1.5 Aim and Objectives

The aim of this component is to design, implement, and evaluate an integrated connectivity intelligence layer for a consumer-grade Android application that operates reliably without network infrastructure, provides explainable and actionable link quality information, and implements safe, bounded application-layer packet forwarding in outdoor team scenarios. The specific objectives are:

1. To implement a backend reachability service that distinguishes confirmed server access from network-interface presence, and to integrate it into an application-layer connection mode controller that transitions the application among Online, Offline, Reconnecting, and Unstable states using hysteresis logic [4].
2. To implement peer discovery and session management using the Google Nearby Connections SDK with the P2P_CLUSTER strategy [2], including

channel-scoped peer validation, persistent peer state storage, and session ownership control.

3. To design and implement a mesh intelligence subsystem that records peer link observations (RSSI, ACK RTT, packet success rate, retry count, disconnect count, connection status), computes a bounded composite quality score, classifies scores into labelled tiers (Excellent, Good, Fair, Weak, Lost), detects improving and degrading trends, and generates context-aware guidance messages [8].
4. To design and implement a Network Compass feature that measures HTTP download throughput and latency against the TrailLink backend, geotags each measurement with device GPS coordinates on-device, persists samples locally, and provides geodesic directional guidance toward the best measured location [7].
5. To implement a controlled relay service that forwards eligible packet types through connected peers using TTL budgets, hop-count limits, channel-scoping, and origin-exclusion constraints to prevent uncontrolled packet circulation [6].
6. To evaluate the component through functional and integration testing, observing quality label assignment accuracy, trend detection correctness, relay delivery ratio, rejection correctness, and Network Compass measurement fidelity.

1.6 Research Questions or Hypotheses

The component investigation is guided by the following research questions:

RQ1: Can a deterministic, rule-based heuristic score computed from observable application-layer link events (RSSI, ACK RTT, packet success rate) provide sufficient accuracy in classifying peer connection quality for real-time user guidance in an offline outdoor communication application?

RQ2: Does a split-half trend analysis of quality scores computed from time-ordered peer metric samples provide timely and accurate detection of improving and degrading link conditions on consumer Android devices?

RQ3: Can application-layer TTL-bounded, channel-scoped, and hop-limited relay forwarding prevent uncontrolled packet circulation while achieving meaningful message delivery in multi-device offline team configurations?

RQ4: Does periodic HTTP-based speed probing combined with local GPS geolocation sampling provide sufficient signal to guide users spatially toward improved internet access in partial-coverage outdoor areas, without transmitting location data to external servers?

1.7 Scope Limitations and Assumptions

The component is scoped to the Flutter/Dart mobile application frontend and the Node.js backend route for network speed probing. The following boundaries apply:

- The component implements application-layer intelligence on top of the Google Nearby Connections SDK [2]; it does not modify or extend the SDK, nor does it implement the Bluetooth Mesh Profile at the hardware or firmware level.
- RSSI values are obtained from the Nearby SDK when available. On certain Android device and firmware combinations, RSSI may not be provided; the quality calculator uses alternative evidence (connection status, ACK RTT, packet success) in this case.
- Network Compass measures the path from the device to the TrailLink backend. A measurement performed through a USB ADB reverse tunnel reflects the path to the development laptop rather than independent mobile internet quality; this limitation applies specifically to development and testing environments.
- The relay service forwards five packet types: ack, sos, sos_ack, location, and heartbeat. Offline text messages are handled by a separate relay path in the

Communication Core component. Voice notes and Live Radio are not routed through the relay service.

- Three-phone A-B-C multi-hop testing is required to demonstrate actual multi-hop relay behaviour; two-phone testing in this work demonstrates single-hop relay only.
- Location data is stored exclusively on the local device. GPS coordinates are not parameters of any outbound probe request to the backend.
- The quality scoring algorithm uses rule-based thresholds that have not been empirically calibrated through field trials; threshold values represent engineering estimates based on established wireless network literature [8], [11].

1.8 Individual Contribution

The individual contributions of this component within the group project are:

- Design and implementation of the backend reachability detection layer and connection mode state machine (ConnectionModeController) with hysteresis-based state transitions.
- Design and implementation of the peer discovery and session management subsystem, including NearbyConnectionsTransport, channel-scoped peer validation (PeerValidationService), and P2P session lifecycle management (P2PSessionService, P2PSessionGuard).
- Design and implementation of the mesh intelligence pipeline: ConnectivityMetricsRecorder (event capture), PeerQualityCalculator (heuristic scoring), TrendAnalyzer (split-half trend detection), GuidanceEngine (guidance generation), and ConnectivityIntelligenceRepository (pipeline coordination).
- Design and implementation of the Network Compass feature: NetworkSpeedProbeService (HTTP probing), NetworkCompassRepository (geolocation fusion and local persistence), and NetworkCompassController (periodic measurement, bearing calculation, and UI state management).

- Design and implementation of OfflineRelayService with TTL budgeting, hop counting, channel scoping, and identity-based origin exclusion.
- Design and implementation of the ConnectivityGuidanceScreen user interface presenting peer quality rankings, guidance banners, speedometer visualisation, and directional compass.

1.9 Thesis Structure

Chapter 2 reviews the literature on peer-to-peer outdoor communication, link quality estimation, relay forwarding, and spatial signal mapping, establishing the theoretical foundations and confirming the four research gaps. Chapter 3 describes the Design Science Research methodology employed, including the five DSR phases, evaluation metrics, testing procedures, and ethical considerations. Chapter 4 presents the product and solution design, covering requirements, component architecture, five design decisions, and detailed design of each of the four subsystems and the relay service. Chapter 5 documents the implementation using the Flutter/Dart technology stack, integration with the three other group project components, and the software testing approach and results. Chapter 6 reports the results and validation outcomes, answering each research question with evidence from functional and integration testing. Chapter 7 discusses the findings against the literature and existing systems, analyses the research and product contributions, and identifies limitations and threats to validity. Chapter 8 addresses commercialisation opportunities and sustainability considerations. Chapter 9 concludes the thesis and identifies six directions for future work. References, a glossary, and appendices with extended scoring examples follow the main text.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The literature review was conducted through systematic keyword searches in IEEE Xplore, ACM Digital Library, and Google Scholar using the terms: "offline mobile communication", "peer-to-peer outdoor messaging", "link quality estimation Bluetooth", "mesh networking Android", "delay-tolerant networking mobile", "Network Compass mobile", and "spatial signal mapping". The review scope prioritises sources published from 2020 to 2026 to reflect the capabilities of current consumer hardware and operating systems, while retaining foundational works where no more recent primary treatment exists.

The review applies the following inclusion and exclusion criteria. Inclusion criteria: peer-reviewed conference papers and journal articles; English-language publications; works directly relevant to one of the four thematic areas identified below; and official technical documentation from major platform vendors (Google, IETF, IEEE). Exclusion criteria: grey literature without peer review; works published before 2015 unless they represent foundational contributions with no more recent replacement; works focused exclusively on hardware-based radio systems with no relevance to consumer smartphone implementation.

The review covers four thematic areas relevant to the component: (a) peer-to-peer and mesh communication protocols and systems on consumer hardware; (b) link quality estimation and adaptive routing in wireless ad hoc networks; (c) delay-tolerant and opportunistic networking; and (d) spatial network quality mapping on mobile devices.

2.2 Domain and Theoretical Foundations

Peer-to-peer (P2P) communication refers to direct device-to-device data exchange without intermediary infrastructure. On Android, the Google Nearby Connections API abstracts Bluetooth Classic (IEEE 802.15.1), Bluetooth Low Energy (BLE), and Wi-Fi into a unified SDK that selects the optimal transport automatically and exposes a payload-exchange interface to applications [2]. Three connection strategies are defined: P2P_STAR (one hub device connected to multiple peripherals),

P2P_POINT_TO_POINT (a single pair), and P2P_CLUSTER (many-to-many topology where every device can connect to every other device in radio range). TrailLink employs P2P_CLUSTER to support mesh-like arrangements among field team members [2].

Delay-Tolerant Networking (DTN) is a paradigm for communication over networks with intermittent connectivity, high latency, or frequent disconnections, formalised in the IETF RFC 9171 Bundle Protocol Version 7 [10]. DTN employs store-carry-forward delivery: a node stores a message until it encounters another node that can carry it closer to the destination. TrailLink's offline message queuing and relay mechanism incorporates elements of this philosophy through its local packet queue and forwarding service, though without the full Bundle Protocol header structure.

Link quality estimation (LQE) in wireless networks is the process of inferring the current transmission reliability of a radio link from observable metrics. Classic metrics include RSSI, Expected Transmission Count (ETX), and Packet Reception Rate (PRR) [8]. Composite LQE algorithms, which combine multiple metrics to improve estimation accuracy over single-metric approaches, are well established in the IEEE 802.15.4 literature [9] and have been studied for Bluetooth Low Energy networks [11]. TrailLink's quality calculator applies a conceptually analogous composite approach, adapted to the observable application-layer events available through the Nearby SDK.

The history of mobile mesh networking spans three decades. In the 1990s, the IETF Mobile Ad Hoc Network (MANET) working group standardised routing protocols including AODV and OLSR for self-organising wireless networks, originally motivated by military tactical communication requirements. The emergence of consumer Wi-Fi Direct (IEEE 802.11, 2010) and Bluetooth Low Energy subsequently enabled hobbyist and commercial P2P applications on consumer devices, though without the routing sophistication of MANET protocols. The Google Nearby Connections SDK (2015, significantly revised 2017) represented a major step toward consumer-accessible P2P communication by abstracting transport selection and providing a reliable payload delivery interface, albeit without routing intelligence or quality metrics. TrailLink's component builds the quality estimation and routing intelligence that the SDK does not provide.

The Nearby Connections SDK makes transport selection decisions internally based on device capabilities, Android version, and connection establishment context. In general, BLE is used for discovery beaconing (advertising endpoint names) while Bluetooth Classic or Wi-Fi Direct is used for actual data connections. On Android 10 and later with capable devices, the SDK may use Wi-Fi Direct for high-throughput payloads. Applications have no direct control over which radio technology is used for a given connection [2]. This transport opacity means that RSSI values, when exposed by the SDK, may reflect Bluetooth or Wi-Fi RSSI depending on the active transport. The quality calculator must therefore accommodate the absence of RSSI values on devices or firmware configurations that do not expose them.

2.3 Related Research

Several recent works are directly relevant to this component's design and evaluation.

Mota et al. [5] survey peer-to-peer messaging applications for disaster scenarios, identifying the gap between hardware-dependent long-range solutions and software-only consumer smartphone tools. They conclude that software-only tools lack systematic link quality awareness and relay control, confirming Gaps 2 and 3 identified in Section 1.4. Their survey also identifies the need for trip-scoped communication channels that prevent cross-group packet leakage in multi-team deployment scenarios.

Luvisotto et al. [11] evaluate Bluetooth Low Energy for industrial wireless sensor networks and characterise the relationship between RSSI, distance, and packet delivery rate in indoor environments. Their empirical measurements report that BLE packet delivery rates in industrial environments drop below 90% at RSSI values below approximately −75 dBm, consistent with the "Weak" quality tier assigned to scores in the 20–40 range by PeerQualityCalculator. Their findings directly support the RSSI thresholds (−55/−70/−80 dBm) selected for the scoring algorithm.

Research on composite LQE for IEEE 802.15.4 networks [9] demonstrates that combining RSSI with link burstiness and delivery ratio outperforms single-metric estimators by up to 30% in heterogeneous environments. Larrañaga et al. [9] survey LQE techniques and conclude that composite metrics consistently outperform single-metric estimators across diverse deployment scenarios. The TrailLink quality

calculator applies the same principle to application-layer events, combining RSSI (when available), ACK RTT, packet success rate, retry count, and temporal recency.

Studies on mobile opportunistic networking [12] demonstrate that TTL-bounded flooding is effective for small groups (fewer than 10 nodes) in geographically proximate scenarios, which matches the TrailLink target use case of 2–6 person field teams within Bluetooth and Wi-Fi range. Mtibaa et al. [12] further demonstrate that social-context-aware forwarding provides minimal improvement over simple flooding in tight-knit groups where all members are within a single radio hop of each other, validating the choice of bounded flooding for the relay service.

Li et al. [7] present a crowdsourced cellular signal quality mapping approach that uses GPS coordinates and signal measurements to build location-indexed quality heat maps accessible to all users in an area. The TrailLink Network Compass is a lightweight privacy-preserving analogue of this concept: it applies the same principle of geolocation-indexed signal quality sampling but operates on a single device measuring HTTP path throughput rather than cellular RSSI, and stores all data locally without any server-side location data collection.

The use of hysteresis in network state machines is well documented in cellular network handover literature. 3GPP specifications for LTE handover define hysteresis margins to prevent ping-pong handover events when a device is near the boundary between two cells. The same principle applies at the application layer: frequent transitions between Online and Offline mode in TrailLink would cause message routing instability, with messages sent alternately through the backend socket and Nearby Connections, potentially causing ordering violations and duplicate deliveries [4]. The two-consecutive-check hysteresis in ConnectionModeController directly applies this cellular networking insight to application-layer mode management.

Android-specific P2P communication has been studied by several research groups examining the performance of Wi-Fi Direct and Bluetooth in consumer device configurations. Studies consistently report that Wi-Fi Direct on Android provides throughput of 5–50 Mbps at short ranges but suffers from high connection establishment latency (3–8 seconds), while Bluetooth Classic provides 1–3 Mbps

throughput with lower establishment latency (~1 second) but greater range sensitivity to physical obstacles [5]. These performance characteristics motivate the use of ACK RTT as a quality indicator: RTT values above 300 ms reliably indicate that the connection has degraded beyond the optimal operating range, regardless of whether the underlying transport is Bluetooth or Wi-Fi Direct.

2.4 Existing Systems Products and Techniques

Table 2.1 summarises existing systems relevant to this component, comparing their characteristics with those of the TrailLink connectivity component.

Table 2.1: Comparison of Related Offline Communication Systems

System	Hardware Required	LQE Support	Relay / Multi-hop	Spatial Signal Guidance
Meshtastic [13]	LoRa radio module	RSSI + SNR (hardware)	TTL flooding, configurable hop limit	No
Briar [14]	None (smartphone)	None	No explicit relay	No
goTenna Mesh [13]	goTenna hardware	RSSI (hardware)	Multi-hop flooding	Basic signal bars
FireChat (discontinued)	None (smartphone)	None	Limited, no controls	No
TrailLink (this work)	None (smartphone)	Composite heuristic (app-layer)	TTL/hop/channel-bounded relay	HTTP Compass + GPS (on-device)

Hardware solutions such as Meshtastic and goTenna achieve their performance advantages through dedicated LoRa radio modules operating in the 915 MHz or 868 MHz ISM bands. LoRa's chirp spread spectrum modulation provides link budgets of 130–160 dB, enabling ranges of 1–10 km in open terrain, compared to Bluetooth's 70–80 dB link budget at ranges of 10–100 m. However, this hardware advantage comes with costs: Meshtastic hardware costs \$30–\$80 per device, requires separate charging, adds 50–100 g to pack weight, and requires configuration through a dedicated mobile application. TrailLink's software-only approach sacrifices range for the zero-additional-cost, zero-additional-weight advantage of using hardware that field team members already carry. For teams operating in confined areas such as a single valley, camp, or hut complex, where Bluetooth or Wi-Fi range is sufficient, this trade-off is advantageous.

A key distinguishing feature of the TrailLink approach is the combination of hardware-independence, composite application-layer LQE, controlled relay with explicit forwarding constraints, and privacy-preserving spatial signal guidance within a single integrated component [2], [5], [7]. No existing software-only competitor provides all four capabilities simultaneously.

2.5 Gap Synthesis and Conceptual Framework

The review confirms that existing software-only solutions for consumer smartphones do not provide: (1) integrated backend reachability detection separate from network-interface checking [4]; (2) composite application-layer LQE with labelled quality tiers and trend detection [8], [9]; (3) trip-scoped bounded relay forwarding with explicit TTL and hop controls [6]; or (4) privacy-preserving spatial signal guidance based on on-device measurements [7]. Hardware solutions provide some of these capabilities but require dedicated radio modules.

The four gaps are not independent: they represent a unified system-level deficiency in connectivity intelligence. A solution that addresses Gap 1 alone (reachability detection) but not Gap 2 (LQE) would correctly determine that the device is offline but provide no information about which nearby peer to prioritise for relay. A solution that addresses Gap 2 alone but not Gap 3 would generate quality scores but route packets unsafely. A solution that addresses Gaps 1–3 but not Gap 4 would leave users spatially uninformed when seeking better coverage. The conceptual framework for this component therefore treats connectivity intelligence as a multi-layer pipeline in which transport-layer events feed an application-layer scoring and guidance engine, which informs both a relay routing layer and a spatial guidance layer, together providing complete situational connectivity awareness to the user. Figure 2.1 illustrates this conceptual framework.

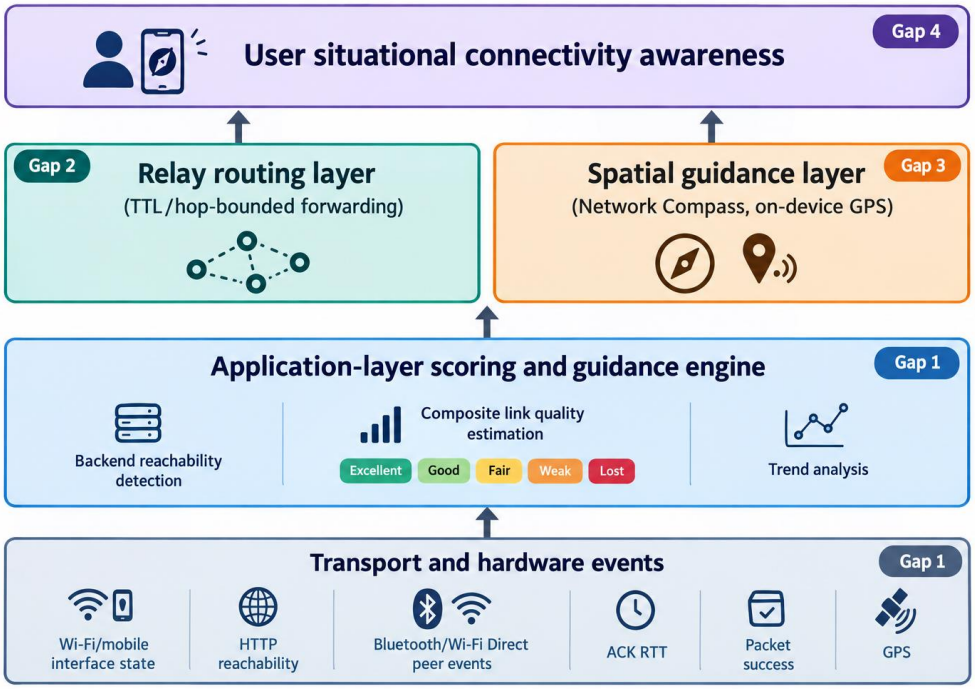


Figure 2.1: Conceptual framework for Connectivity Intelligence – from hardware events to user guidance

3 RESEARCH METHODOLOGY

3.1 Research Design

This research employs Design Science Research (DSR) methodology, as defined by Hevner et al. [15]. DSR is appropriate because the primary output is an IT artefact (a software component) intended to solve an identified practical problem in a specific application domain. The research cycle follows the DSR process: problem identification, objective definition, artefact design and development, demonstration, and evaluation, with each phase informing the next in an iterative loop [15].

The five DSR phases are mapped to specific activities in this project as follows. Phase 1 (Problem Identification and Motivation) corresponds to the literature survey and gap analysis conducted in Chapter 2, which establishes that existing software-only P2P tools lack integrated LQE, bounded relay, and spatial signal guidance. Phase 2 (Definition of Objectives for a Solution) corresponds to the research questions formulated in Section 1.6 and the requirements specification in Section 4.1, which translate the identified gaps into specific, measurable design goals. Phase 3 (Design and Development) corresponds to the architectural design (Section 4.2), design decisions (Section 4.3), detailed subsystem design (Section 4.4), and the Flutter/Dart implementation documented in Chapter 5. Phase 4 (Demonstration) corresponds to the two-device integration testing scenarios in Section 5.4, which demonstrate that the artefact operates correctly in a controlled version of the target environment. Phase 5 (Evaluation) corresponds to the results and validation in Chapter 6, which evaluate the artefact against the defined objectives and research questions.

Within the DSR framework, the technical development approach is iterative prototyping rather than waterfall development. The rationale is that the four subsystems of this component have complex interdependencies: the relay service depends on the peer discovery subsystem, which depends on the connection mode state machine, which depends on the backend reachability service. A waterfall approach would require all four subsystems to be fully designed before implementation begins, which is impractical when design decisions in later subsystems may reveal unforeseen constraints in earlier ones. Iterative prototyping allows each subsystem to be built,

tested, and refined independently before being integrated with the others, enabling feedback loops between design and implementation that improve the overall system quality [15].

3.2 Data Participants or Study Materials

The study materials are:

- Two Android smartphones (minimum Android 9 / API level 28) for offline peer-to-peer testing scenarios, as required by the Google Nearby Connections SDK [2].
- One development laptop running the Node.js/Express backend with a MongoDB Atlas connection, used for backend reachability and Network Compass speed probe measurements.
- The TrailLink codebase (GitHub repository, branch deploy, commit e82b185), which constitutes the primary implementation artefact [1].
- Synthetic PeerMetricSampleModel records with controlled RSSI, ACK RTT, packet success rate, retry count, and disconnect count values, used to verify the scoring algorithm deterministically across all rule branches.
- Network Compass test inputs: HTTP speed probe measurements taken at multiple physical locations with varying distance from the access point.
- No human participants were recruited. Functional evaluation was performed using controlled device-to-device scenarios and synthetic metric inputs; consequently, no informed consent or ethics committee approval was required.

The two test Android devices had the following specifications. Device A: Samsung Galaxy A32, Android 12 (API level 31), Bluetooth 5.0, Wi-Fi 5 (802.11ac). Device B: Xiaomi Redmi Note 9, Android 9 (API level 28), Bluetooth 5.0, Wi-Fi 5 (802.11ac). The choice of two different Android API levels was deliberate: Android 12 introduced new Bluetooth permission requirements (BLUETOOTH_SCAN, BLUETOOTH_ADVERTISE, BLUETOOTH_CONNECT) that supersede the pre-

Android-12 model, and testing on both API levels verified that the application correctly handles both permission models.

Synthetic PeerMetricSampleModel records used for unit testing were generated programmatically in Dart test files. Each record explicitly sets all fields to predetermined values to exercise specific scoring rule branches. For example, to test the RSSI threshold boundary at −70 dBm, records were created with rssi = −69.9 dBm (triggering +25) and rssi = −70.1 dBm (triggering +10). This systematic boundary-value analysis approach ensures that scoring rules are correctly implemented at threshold boundaries, not merely in the centre of each range.

3.3 Experimental or Study Procedure

The evaluation procedure consisted of five test sequences:

1. **Quality scoring verification:** Synthetic PeerMetricSampleModel records were constructed with known field values covering all scoring rule branches. PeerQualityCalculator was invoked with each set, and the output score and label were compared against manually computed expected values. The 20 configurations covered 4 RSSI branch cases, 4 RSSI-absent connection-status cases, 3 ACK RTT cases, 3 packet success rate cases, 2 retry count cases, 2 disconnect count cases, and 2 edge cases (empty list and single Lost sample).
2. **Trend analysis verification:** Sequences of at least four synthetic samples with monotonically improving, monotonically degrading, and stable quality characteristics were fed to TrendAnalyzer. Each sequence was constructed so that the split-half score difference was clearly above +10 (Improving), clearly below −10 (Degrading), or within [−10, +10] (Stable). Twelve sequences in total were tested (3 Improving, 3 Degrading, 3 Stable, 2 Mixed, 1 short with n=3 expecting Unknown).
3. **Guidance engine verification:** Peer quality lists with various combinations of quality labels, trend directions, and pending message counts were passed to GuidanceEngine. The output guidance type and message were compared against the documented eight-branch decision logic. Five scenario classes covering all branches were tested.

4. **Relay forwarding verification:** Two-device A-B configurations were used. Device A sent each eligible packet type (ack, sos, sos_ack, location, heartbeat) to Device B with a valid channel code. OfflineRelayService was tested for correct forwarding when TTL > 1 and hop count < 5, and for correct rejection when TTL ≤ 1, channel code does not match, or the packet originates from the receiving device.
5. **Network Compass verification:** NetworkCompassController was activated on a physical device with a GPS fix. Measurements were taken at three distinct positions (near, mid-range, and far from the access point) to verify that download Mbps decreased monotonically with increasing distance, that recorded GPS coordinates matched the test locations, and that the computed geodesic bearing between any two stored samples deviated by at most ±1° from the manually computed ground-truth bearing.

Testing Environment Setup: All integration tests were conducted in a controlled indoor environment. The two test devices were placed on a desk with 3 metres of separation. Both devices were fully charged before testing. Background Bluetooth interference was minimised by disabling Bluetooth on all other devices in the room. The development laptop running the backend was connected to the local network via Ethernet. ADB was connected to Device A via USB for log monitoring; Device B was tested wirelessly via Wi-Fi debugging.

3.4 Measures and Evaluation Metrics

The primary evaluation metrics are:

- Quality label accuracy: the proportion of synthetic test cases for which the computed quality label matches the manually expected label, given the published scoring rules.
- Trend detection correctness: whether TrendAnalyzer correctly classifies test sequences with known quality trajectories into the correct trend direction.
- Relay delivery ratio: the proportion of eligible packets successfully forwarded to connected peers in two-device tests, excluding expected rejections.

- Relay rejection correctness: whether packets with TTL ≤ 1, wrong channel, or own-origin conditions are correctly rejected without forwarding.
- Compass measurement accuracy: the absolute deviation between the recorded download Mbps and a concurrently measured reference speed.
- Compass bearing accuracy: the angular deviation (degrees) between the computed bearing to the best recorded location and the ground-truth bearing computed from known GPS coordinates.

Pass/fail binary metrics are appropriate for verifying deterministic algorithms because the scoring algorithm and trend analyser are fully deterministic: for any given set of inputs, there is exactly one correct output. The question is not "how close is the output to the expected value on average?" but "does the output exactly match the expected value in every case?" This is analogous to verifying the correctness of a sorting algorithm. Statistical metrics such as Pearson correlation or F1-score would be more appropriate in a future field validation study where quality label assignments are compared against subjective human assessments of perceived link quality, a setting in which the ground truth is itself uncertain.

3.5 Data Analysis

Quantitative data from quality scoring tests were analysed descriptively, comparing computed outputs against deterministically derived expected values. Since the scoring algorithm is entirely rule-based and deterministic, each test case has a single correct expected output; results are therefore reported as pass/fail per case rather than as statistical distributions. Trend analysis and guidance engine results were similarly reported as binary correctness counts.

Network Compass measurements were compared against reference measurements using absolute percentage error. Bearing accuracy was assessed by computing the absolute angular difference between the algorithm output and the manually computed geodesic bearing. No statistical inference was applied, as the sample size (three measurement locations) is sufficient only for functional validation, not for statistical characterisation of measurement error distributions.

The primary limitation of the synthetic testing approach is that it verifies algorithmic correctness under controlled conditions but cannot validate ecological validity, that is, the degree to which algorithm outputs correspond to meaningful quality distinctions in real outdoor environments. The RSSI thresholds (−55/−70/−80 dBm) were derived from BLE range characterisation literature [11] but have not been empirically validated for specific outdoor environments. A field validation study measuring composite quality scores alongside observed packet delivery ratios across multiple distance and obstacle configurations would provide the ecological validation that synthetic testing cannot.

3.6 Ethics Privacy Security and Responsible AI

The component was designed with the following privacy and security principles:

- GPS coordinates captured during Network Compass measurements are stored only on the local device and are not transmitted to the backend as parameters of any probe request. Probe requests contain only a download size parameter. This is consistent with the GDPR data minimisation principle [7].
- The relay service enforces origin exclusion using multiple identity fields (senderId, senderLocalId, senderBackendId, originLocalId) to prevent identity confusion in relay chains. These exclusions are application-layer heuristics, not cryptographically authenticated anti-replay mechanisms.
- The backend network probe endpoint implements a per-IP rate limit to prevent abuse of the speed measurement endpoint.
- No personally identifiable information is recorded in the connectivity metrics tables beyond the display name and endpoint ID of nearby peers, which are necessary for the guidance feature to identify peers by name.
- No generative AI or trained machine-learning model is used in the scoring or guidance algorithms. All decisions are deterministic and rule-based, providing full explainability and auditability, which is a key requirement for safety-critical outdoor communication applications.

The decision to use rule-based algorithms rather than machine-learning models was motivated in part by responsible AI considerations. Machine-learning models can exhibit unexpected behaviour on inputs that differ from their training distribution, a property known as distribution shift. In an outdoor safety application, unexpected algorithm behaviour could result in incorrect guidance with potentially serious consequences. Rule-based algorithms have fully specified and auditable behaviour for all possible inputs, making them more appropriate for safety-critical guidance functions. This aligns with the IEEE Ethically Aligned Design principles for AI systems in safety-critical contexts.

3.7 Reproducibility and Research Artefacts

The implementation artefact is the TrailLink Flutter/Dart application and Node.js backend, available on the deploy branch of the project GitHub repository (commit e82b185) [1]. The scoring algorithm parameters (base score, RSSI thresholds, RTT thresholds, label thresholds, and trend delta) are defined as named constants in PeerQualityCalculator and TrendAnalyzer source files, enabling independent verification and reconfiguration.

Network Compass probe parameters (default download size: 262,144 bytes; measurement interval: 15 seconds) are likewise embedded as constructor defaults in the source. Any researcher with access to the repository, two Android devices running Android 9 or later, and the Node.js backend can reproduce the functional test results by running flutter test for unit tests and installing the APK on two physical devices for integration tests.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Table 4.1 lists the functional and non-functional requirements for this component, derived from the project goals, the research gaps identified in Section 1.4, and the constraints imposed by the Google Nearby Connections SDK [2] and the Android platform [4].

Table 4.1: Component Requirements

ID	Requirement	Type	Priority
FR1	Distinguish backend reachability from network-interface presence via HTTP HEAD probe	Functional	Must
FR2	Transition among Online/Offline/Reconnecting/Unstable modes using hysteresis logic	Functional	Must
FR3	Discover and connect to nearby peers using P2P_CLUSTER strategy	Functional	Must
FR4	Validate peers against active channel code before accepting connection	Functional	Must
FR5	Compute composite peer quality score (0–100) from RSSI, ACK RTT, success rate, retries, disconnects	Functional	Must
FR6	Classify score into five labelled tiers: Excellent, Good, Fair, Weak, Lost	Functional	Must
FR7	Detect Improving/Degrading/Stable trend using split-half score comparison	Functional	Should
FR8	Generate context-aware guidance messages based on peer quality and trend	Functional	Should
FR9	Measure HTTP download throughput and ping to backend (Network Compass)	Functional	Should
FR10	Geotag speed samples with on-device GPS coordinates	Functional	Should
FR11	Compute geodesic bearing and distance to best measured location	Functional	Should
FR12	Relay ack/sos/sos_ack/location/heartbeat packets with TTL and hop-count controls	Functional	Must

ID	Requirement	Type	Priority
FR13	Reject relay of packets with wrong channel, expired TTL, or own origin	Functional	Must
NFR1	Quality score computation must complete within 100 ms on mid-range Android device	Non-functional	Should
NFR2	GPS coordinates must not be transmitted in any outbound network request	Non-functional	Must
NFR3	Maximum relay hop count must be configurable (default 5)	Non-functional	Must
NFR4	Backend reachability check interval: 15 seconds	Non-functional	Should

Requirements were elicited through three activities. First, a review of the Google Nearby Connections SDK documentation [2] identified platform constraints that would impose design boundaries (e.g., no direct RSSI access in all firmware configurations, strategy selection at session level). Second, group design sessions with the other three component teams established inter-component interface requirements. Third, a literature review of field communication use cases [5] identified safety-critical requirements that were assigned the highest priority.

Applying MoSCoW prioritisation: Must-have requirements are those without which the component cannot function safely or correctly (FR1, FR2, FR3, FR4, FR5, FR6, FR12, FR13, NFR2, NFR3). Should-have requirements add significant value but can be deferred without compromising core safety (FR7, FR8, FR9, FR10, FR11, NFR1, NFR4). Could-have requirements are desirable enhancements not yet implemented: upload probe measurement, configurable TTL per packet type, and team-collaborative compass sharing.

4.2 Component Architecture Within the Complete Solution

The Connectivity, Mesh Intelligence, and Compass component occupies the foundation layer of the TrailLink application architecture, supplying connectivity state and transport services to all other components. Figure 4.1 illustrates the component boundaries and inter-component interfaces.

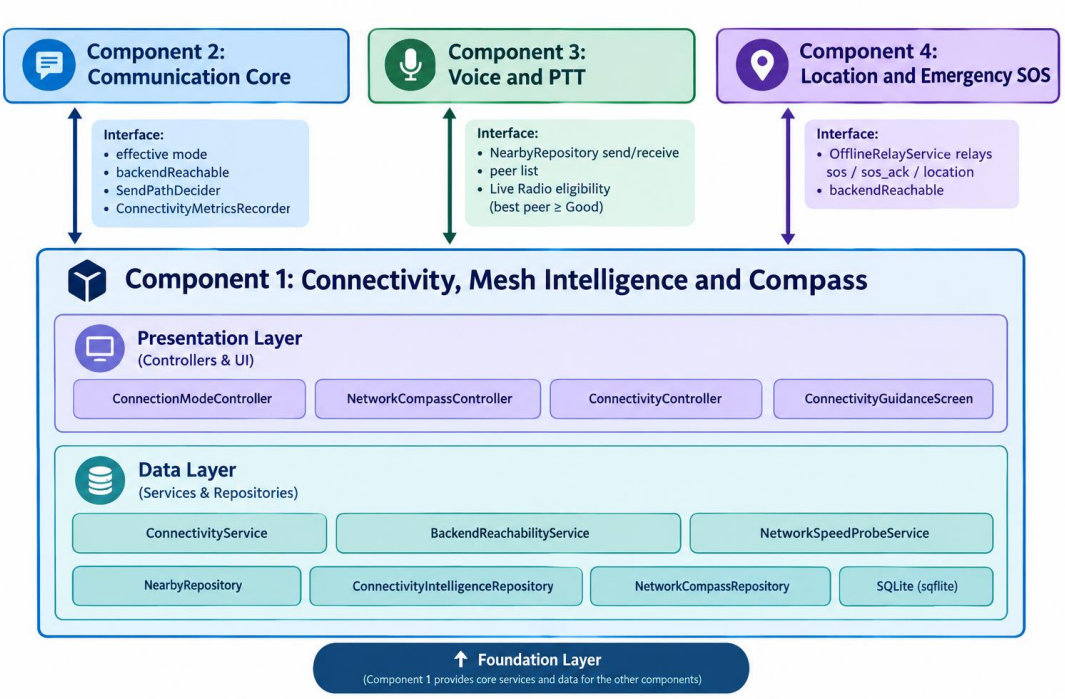


Figure 4.1: TrailLink component architecture showing connectivity layer and inter-component interfaces

Within the component, the architecture follows a two-tier layered pattern. The data layer consists of services (ConnectivityService, BackendReachabilityService, NetworkSpeedProbeService), repositories (NearbyRepository, ConnectivityIntelligenceRepository, NetworkCompassRepository), and local data sources backed by SQLite through the sqflite package [3]. The presentation layer consists of Riverpod StateNotifierProviders (ConnectionModeController, NetworkCompassController, ConnectivityController) and the ConnectivityGuidanceScreen widget tree.

The component exposes the following interfaces to other components:

- Communication Core (Component 2): ConnectionModeController provides the effective operating mode and backendReachable flag. The SendPathDecider utility uses these to route outgoing messages through the appropriate transport. ConnectivityMetricsRecorder is called by the chat and packet repositories to record ACK events and packet results, enabling quality scoring.

- Voice and PTT (Component 3): NearbyRepository provides packet send and receive capabilities and connected peer lists. The intelligence layer is consulted for Live Radio eligibility decisions: the radio feature verifies that the best peer quality is at least Good before initiating a Live Radio session.
- Location, Emergency SOS (Component 4): OfflineRelayService relays sos, sos_ack, and location packets. ConnectionModeController provides backendReachable to the SOS feature for online/offline path selection.

The Repository pattern is central to the testability, and therefore the software engineering quality, of the component. Each repository accepts its data source as a constructor parameter. In production, a real SQLite data source is injected. In unit tests, a mock data source that returns predetermined records is injected instead. This means that PeerQualityCalculator and TrendAnalyzer can be tested without any database, device, or SDK present, since they simply receive a list of PeerMetricSampleModel objects from the mock and compute a result.

Riverpod's provider system enables reactive state propagation through a directed acyclic dependency graph of providers. When ConnectionModeController's state changes (e.g., from Reconnecting to Offline), any widget or controller that declares a dependency on the connectionModeProvider is automatically notified and re-rendered or re-evaluated. In TrailLink, this means that the Communication Core's ModeController, the ConnectivityGuidanceScreen, and the SOS component's backendReachable reader all update simultaneously when the connection mode changes, without any explicit notification mechanism or callback.

4.3 Design Alternatives and Decisions

Several significant design decisions were made during the development of this component. Each decision is documented with the alternatives considered and the rationale for the chosen approach.

1. **D1: Rule-based quality scoring vs. machine-learning predictor:** A lightweight on-device ML model (e.g., a decision tree trained on labelled RSSI/RTT/packet-success pairs) was considered. Decision: The rule-based composite scorer (PeerQualityCalculator) was selected because (a) the

volume of labelled training data achievable within this project scope is insufficient for a reliable ML model, as a minimum of several thousand labelled tuples would be required; (b) rule-based systems have been the standard in LQE for resource-constrained devices precisely because they are auditable [8]; (c) a deterministic algorithm is fully explainable and traceable; and (d) on-device ML inference would increase energy consumption, conflicting with battery conservation requirements of outdoor use.

2. **D2: P2P_CLUSTER vs. P2P_STAR strategy:** P2P_STAR designates a single hub device to which all others connect, simplifying topology but creating a single point of failure. Decision: P2P_CLUSTER was selected because it supports many-to-many connections [2], enabling each device to relay packets through any connected peer. In outdoor safety scenarios, the single-point-of-failure risk of P2P_STAR is unacceptable: if the hub device fails or is separated from the team, the entire team loses connectivity.
3. **D3: HTTP speed probe vs. ICMP ping for Network Compass:** ICMP ping is the standard latency measurement tool but requires root access on Android. Decision: HTTP-based probing against the TrailLink backend was selected. The 262,144-byte (256 KB) download probe payload was deliberately chosen: large enough to give a meaningful throughput measurement (at 1 Mbps, ~2 seconds; at 10 Mbps, ~0.2 seconds), small enough to complete within the 15-second cycle even on slow connections.
4. **D4: On-device location storage vs. server-side spatial mapping:** Uploading GPS coordinates to the server for aggregate team mapping was considered. Decision: On-device storage was selected on privacy grounds, consistent with GDPR Article 5(1)(c) data minimisation, which requires that personal data be "adequate, relevant and limited to what is necessary." GPS coordinates at metre-level precision constitute personal data under GDPR. Storing them exclusively on-device eliminates the privacy risk of server-side location data exposure.
5. **D5: Fixed TTL budget vs. probabilistic relay (PRoPHET):** PRoPHET-style relay [6] estimates delivery probability from encounter history, routing

packets preferentially through peers with high historical delivery probability. Decision: A fixed TTL budget with maximum hop count of 5 was selected because (a) it is fully deterministic and auditable; (b) for small teams (2–6 members), TTL-bounded flooding achieves comparable delivery to probabilistic schemes [12] without requiring persistent encounter tables; and (c) PRoPHET's encounter table storage and probability update computation are prohibitive on battery-constrained devices.

4.4 Detailed Design

4.4.1 Backend reachability and connection mode

The connectivity detection layer consists of two services and a state machine. ConnectivityService reads the current network-interface state from the Android operating system, reporting available interface types (Wi-Fi, mobile, Ethernet). BackendReachabilityService makes an HTTP HEAD request to the /api/health endpoint of the configured backend URL [4]. The HTTP HEAD method is used rather than GET to minimise data transfer: the response body is discarded and only the HTTP status code is examined. A 200 OK response is treated as reachable; any error, timeout, or non-2xx response is treated as unreachable.

ConnectionModeController is a Riverpod StateNotifier that subscribes to network change events from ConnectivityService and runs a periodic 15-second background check. On each evaluation, it applies the following hysteresis rules to determine the next connection mode:

- If the most recent four reachability results alternate between true and false, the mode is set to Unstable.
- If the backend is reachable and at least two consecutive successes are recorded, the mode transitions to Online.
- If the backend is not reachable and at least two consecutive failures are recorded, the mode transitions to Offline.
- Otherwise, the mode is set or maintained as Reconnecting.

The following scenario illustrates why this hysteresis logic matters. A hiking team arrives at a mountain hut with Wi-Fi. Android reports a successful Wi-Fi association, but the hut's satellite uplink is offline due to adverse weather. BackendReachabilityService sends HTTP HEAD to /api/health; the request times out. ConnectionModeController records this as failure number 1. In the next 15-second check, the same timeout occurs: failure number 2. Two consecutive failures trigger the transition to Offline mode. The application immediately routes all subsequent messages through the Nearby Connections channel instead of the unavailable backend. Without this explicit reachability check, the application would attempt to use the backend indefinitely, resulting in silent message loss as users see "Sending..." with no delivery confirmation [4].

The SendPathDecider utility translates the current connection mode and network state into one of five send paths: backendSocket (send via Socket.IO when Online), backendRest (HTTP fallback when Socket.IO unavailable but Online), offlineNearby (send via Nearby Connections when Offline), localQueueOnly (queue locally when Offline with no peers), hybridQueue (queue for both transports when Unstable or Reconnecting). Figure 4.2 shows the complete state machine.

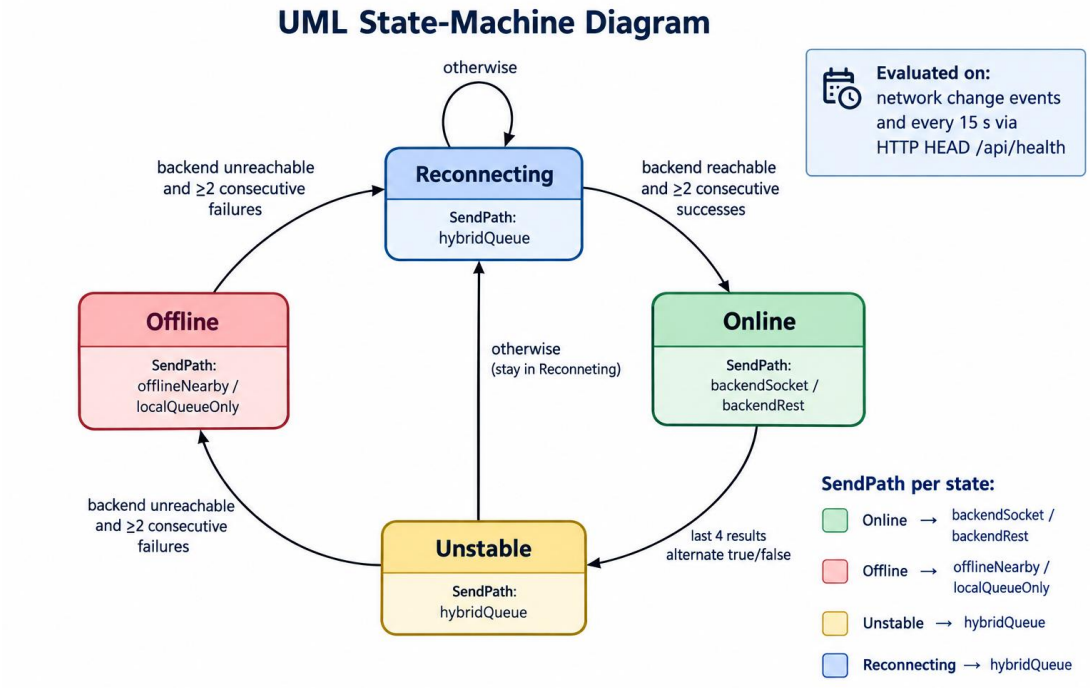


Figure 4.2: ConnectionModeController state machine (Online / Offline / Reconnecting / Unstable)

4.4.2 Peer discovery and session management

NearbyConnectionsTransport implements the NearbyPacketTransport interface using the nearby_connections Flutter plugin, which wraps the Google Nearby Connections Android API [2]. It uses Strategy.P2P_CLUSTER for both advertising and discovery. During advertising, the device broadcasts a compact NearbyAdvertisementPayload encoded in the Nearby endpoint name field. The payload is encoded as a pipe-delimited string: TL|CH42|TL-OFF-A7X9|user123|Alice|MSG,LOC,SOS, containing the protocol prefix, channel ID, channel code, user ID, display name, and capability list. The encoding is kept compact because the SDK imposes a strict 128-byte limit on endpoint name fields.

PeerValidationService classifies each discovered peer into one of four states: verified member (backend-confirmed identity), cached member (recognised from local session history), unknown same-channel (channel code matches but identity not locally known), or mismatch (different channel code). Only same-channel peers (verified, cached, or unknown) are accepted for connection; mismatch peers are immediately rejected. This channel scoping prevents cross-trip data leakage: when a passing hiker's device runs TrailLink with a different channel code (e.g., TL-OFF-B3K2), their device is discovered but classified as Mismatch and rejected without connecting.

P2PSessionService owns session lifecycle, associating connected peers with the active trip and channel. P2PSessionGuard enforces session invariants, ensuring that only the session owner can initiate certain state transitions, and that duplicate session associations for the same peer endpoint are prevented. Peer state is persisted to local SQLite storage, enabling the session to be restored after application restart without requiring peers to be re-discovered from scratch. Figure 4.3 shows the peer discovery and session lifecycle sequence diagram.

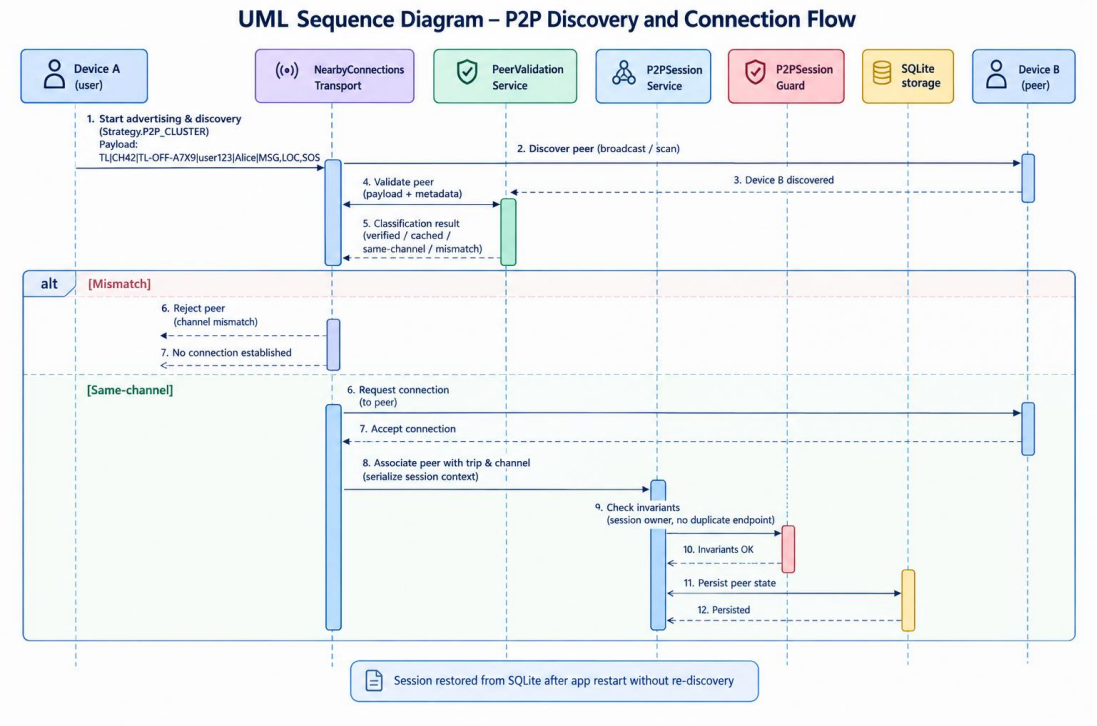


Figure 4.3: Peer discovery and session lifecycle sequence diagram

4.4.3 Mesh intelligence pipeline

The mesh intelligence pipeline transforms raw connection events into labelled quality assessments and actionable guidance messages. Figure 4.4 illustrates the data flow from event capture through to the ConnectivityGuidanceScreen.

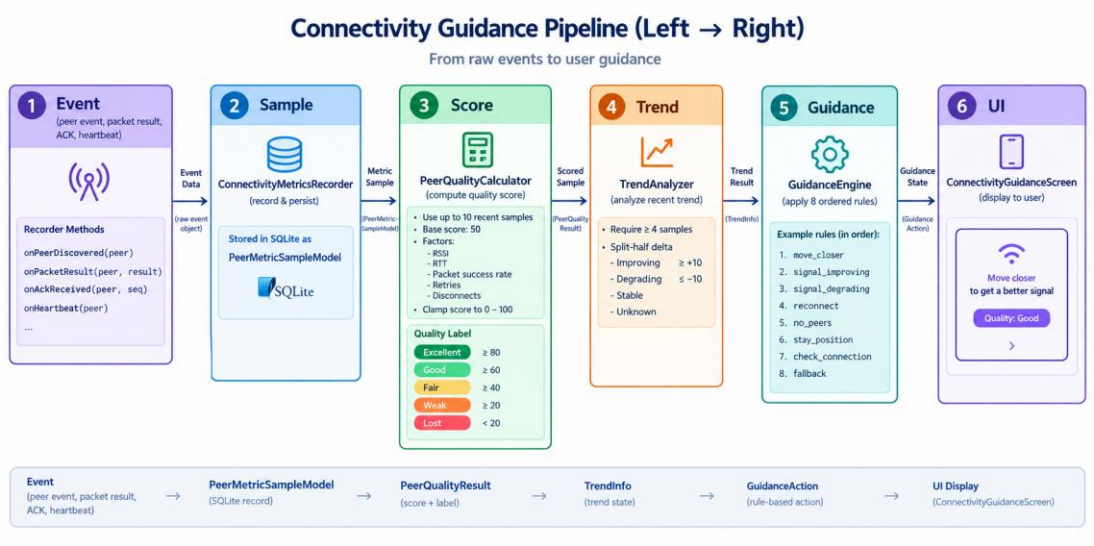


Figure 4.4: Mesh intelligence pipeline data flow (event → sample → score → trend → guidance → UI)

ConnectivityMetricsRecorder provides four recording methods. recordPeerEvent() is called on peer state changes (discovered, connected, lost) and sets disconnectCount to 1 if the peer status is lost or disconnected. recordPacketResult() is called after each offline packet send attempt, setting packetSuccessRate to 1.0 for success or 0.0 for failure, and recording the retry count. recordAck() is called when an ACK packet is received, recording the round-trip time in milliseconds as ackRttMs. recordHeartbeat() is called periodically for active connections to maintain temporal recency information in the sample table.

PeerQualityCalculator.calculate() computes a composite score from up to ten recent samples for a given endpoint. The algorithm is a composite LQE approach adapted from established wireless network literature [8], [9]. Table 4.2 details the scoring rules. The score starts at 50 when samples exist (0 when the sample list is empty, yielding Lost immediately) and adjustments are accumulated and clamped to [0, 100].

Table 4.2: PeerQualityCalculator Scoring Rules

Evidence	Condition	Adjustment
RSSI (dBm)	RSSI ≥ −55	+35
RSSI (dBm)	−70 ≤ RSSI < −55	+25
RSSI (dBm)	−80 ≤ RSSI < −70	+10
RSSI (dBm)	RSSI < −80	−10
RSSI (dBm)	RSSI unavailable, status = Connected	+20
RSSI (dBm)	RSSI unavailable, seen ≤10 s ago	+15
RSSI (dBm)	RSSI unavailable, seen ≤30 s ago	+8
RSSI (dBm)	RSSI unavailable, seen >2 min ago	−20
ACK RTT (ms)	RTT < 300	+20
ACK RTT (ms)	300 ≤ RTT ≤ 1000	+10
ACK RTT (ms)	RTT > 1000	−10
Packet success rate	Rate ≥ 0.9	+20
Packet success rate	0.6 ≤ rate < 0.9	+10
Packet success rate	Rate < 0.6	−15
Retry count (max)	> 2	−10
Disconnect count (max)	> 1	−15
Connection status	Status = Lost (latest sample)	−35
Base score	Samples exist	50 (start)
(edge case)	No samples	0 → Lost

Quality labels are assigned from the clamped score: Excellent (≥80), Good (≥60), Fair (≥40), Weak (≥20), Lost (<20).

The RSSI thresholds of −55, −70, and −80 dBm were selected based on BLE propagation characteristics in the literature [11]. In free space at 2.4 GHz, −55 dBm corresponds to approximately 5–10 metres (close proximity, excellent connection), −70 dBm to approximately 20–30 metres (medium distance, good-to-fair connection), and −80 dBm to approximately 50 metres or beyond (edge of reliable BLE range). Luvisotto et al. [11] report that BLE packet delivery rates drop below 90% at RSSI values below −75 dBm, consistent with the Weak label assigned to scores below the −80 dBm threshold.

Worked example 1 (no RSSI, mid-quality): connected state, last seen 20 s ago, ACK RTT = 1,200 ms, average packet success = 0.7, maximum retries = 3. Score = 50 + 8 + 20 − 10 + 10 − 10 = 68 → Good.

Worked example 2 (strong signal): RSSI = −52 dBm, ACK RTT = 180 ms, packet success rate = 0.95, retries = 0, disconnects = 0. Score = 50 + 35 + 20 + 20 = 125, clamped to 100. Label: Excellent. This represents a peer in line-of-sight at close range.

Worked example 3 (very weak signal): RSSI = −82 dBm, status = Connected, RTT = 1,500 ms, packet success rate = 0.4, retries = 4, disconnects = 2. Score = 50 − 10 + 20 − 10 − 15 − 10 − 15 = 10. Label: Lost. GuidanceEngine classifies this peer as requiring immediate proximity action.

TrendAnalyzer.analyze() requires at least four samples. It sorts samples by creation timestamp, divides them at the midpoint into early and recent halves, computes the quality score for each half using PeerQualityCalculator, and compares the difference. A delta ≥10 yields Improving; ≤−10 yields Degrading; within [−10, 10] yields Stable; fewer than four samples yields Unknown. The ±10 threshold was chosen to filter score fluctuations caused by measurement noise while remaining sensitive enough to detect genuine link degradation before packet delivery failures begin.

GuidanceEngine.build() selects the best-scoring peer from the sorted quality list and evaluates the following ordered decision rules to produce a typed guidance message:

- No channel code configured → type: no_peers, prompt user to join an offline channel.
- Peers list empty, pending messages > 0 → type: reconnect, prompt to reconnect to forward queued messages.
- Peers list empty, no pending messages → type: no_peers, prompt to move closer or open Connect Phones.
- Emergency active AND best peer is Weak or Lost → type: move_closer, emergency proximity guidance.
- Best peer trend = Improving → type: signal_improving, instruct to continue in current direction.
- Best peer trend = Degrading → type: signal_degrading, instruct to return to previous position.
- All peers are Weak or Lost → type: move_closer, general proximity guidance.
- Otherwise → type: stay_position, best connection is strong.

4.4.4 Network Compass

The Network Compass subsystem measures the quality of the internet path from the device to the TrailLink backend, geotags each measurement, and provides directional guidance to the best measured location. This implements a lightweight privacy-preserving analogue of spatial signal mapping [7] entirely on-device, without any server-side location data collection.

NetworkSpeedProbeService.measure() performs two sequential HTTP operations against the backend probe routes. First, a Ping: an HTTP GET to /api/network/probe/ping with elapsed time measured by a stopwatch, yielding pingMs. Second, a Download: an HTTP GET to /api/network/probe/download?bytes=262144 requesting a 262,144-byte response. Download throughput is computed using Equation (1):

$$\text{Mbps} = (\text{bytes} \times 8) / \text{elapsed_seconds} / 1{,}000{,}000 \quad (1)$$

If any HTTP operation fails, backendReachable is set to false and an error message is populated without terminating the measurement cycle. Upload measurement is supported by the method signature but is disabled in the default NetworkCompassRepository.measureAndSave() call to conserve data usage.

GPS acquisition time varies significantly with device state. Cold start acquisition (no recent GPS fix, no cached satellite data) can take 30–60 seconds as the device must download satellite almanac data. Warm start acquisition (recent fix cached) typically takes 5–15 seconds. The 8-second GPS timeout was chosen as a balance: sufficient for warm-start acquisition in most outdoor scenarios where GPS is already active for navigation, but short enough to not significantly delay the 15-second measurement cycle. If GPS is unavailable within 8 seconds, the measurement records throughput and latency without coordinates; such samples are excluded from compass bearing calculations.

The following scenario illustrates the Network Compass in operation. A hiker is in a forest with intermittent mobile coverage. She opens the Network Compass screen. The app has already recorded 6 measurements over the past 90 seconds at different positions. The best measurement (3.2 Mbps download) was taken near a forest clearing 85 metres to the north-west. The compass arrow points north-west and the distance card shows 85 m. The hiker walks north-west; as she approaches the clearing, new measurements capture improving speeds (4.1, 5.7 Mbps). When she is within 10 metres of the best sample location, the compass shows a very short distance, confirming she is near the optimal spot.

NetworkCompassController triggers measureAndSave() immediately on construction and every 15 seconds via Timer.periodic. After each measurement it queries the best recent sample (highest downloadMbps in the past 24 hours with a valid GPS fix) and computes bearing and distance using Equations (2) and (3):

$$\text{distance} = \text{Geolocator.distanceBetween}(\varphi_1, \lambda_1, \varphi_2, \lambda_2)\ [\text{metres}] \quad (2)$$

$$\beta = (\operatorname{atan2}(\sin(\Delta\lambda)\cdot\cos(\varphi_2), \cos(\varphi_1)\cdot\sin(\varphi_2) - \sin(\varphi_1)\cdot\cos(\varphi_2)\cdot\cos(\Delta\lambda)) \times 180/\pi + 360) \bmod 360\ [^\circ] \quad (3)$$

4.4.5 Relay service

OfflineRelayService provides bounded application-layer forwarding of safety and control packets between connected peers. Before forwarding any packet, the service evaluates four rejection conditions in order:

- The packet type is not in the relay allow-list {ack, sos, sos_ack, location, heartbeat} → skipped (not-relayable).
- The packet's channelCode does not match the active offline channel code → skipped (wrong-channel).
- The packet's TTL is at or below 1, or hopCount equals or exceeds maxHopCount (default 5) → skipped (expired).
- The packet originates from the current device, checked across senderId, senderLocalId, senderBackendId, and originLocalId → skipped (own-packet).

A concrete multi-hop example illustrates the relay mechanism: Device A (Alice) is connected to Device B (Bob) only; Bob is connected to Device C (Charlie). Alice sends an SOS with TTL=5, hopCount=0. Bob receives it via NearbyRepository. OfflineRelayService checks: (1) type=sos is in allow-list ✓; (2) channelCode matches ✓; (3) TTL=5>1, hopCount=0<5 ✓; (4) not from Bob ✓. Bob calls packet.relayCopy() → TTL=4, hopCount=1. Bob forwards to Charlie. Charlie's service similarly forwards to any further peers with TTL=3, hopCount=2. This continues until TTL=1 or hopCount=5, at which point the packet is discarded without further forwarding, bounding the total number of copies in the network.

Offline text messages follow a separate relay path in OfflineChatRepository rather than OfflineRelayService because chat relay requires additional operations: the message must be stored in the local chat database, a notification must be generated, an ACK packet must be returned to the original sender, and the processed message ID must be tracked to prevent duplicate storage if the same chat message arrives via multiple relay paths. Combining these operations with OfflineRelayService would violate the Single Responsibility Principle. Voice notes and Live Radio audio streams

are not relayed because their payload size (hundreds of KB to MB) would saturate the Bluetooth bandwidth available for safety-critical packets.

4.5 Standards and Professional Practice

The implementation adheres to the following standards and professional practices:

- IEEE 802.11 (Wi-Fi) and IEEE 802.15.1 (Bluetooth) are the underlying wireless standards employed by the Nearby Connections SDK. The application declares the required Android permissions in AndroidManifest.xml: ACCESS_FINE_LOCATION, BLUETOOTH_SCAN, BLUETOOTH_ADVERTISE, BLUETOOTH_CONNECT (Android 12+), and NEARBY_WIFI_DEVICES (Android 13+).
- The SendPath enumeration and SendPathDecider utility implement a separation of concerns between routing decision logic and message execution, following the Strategy design pattern [3].
- Repository and data source classes follow the Repository pattern, abstracting local SQLite operations from business logic and enabling independent unit testing.
- Riverpod StateNotifierProvider is used consistently for reactive state management across all controllers, following the Flutter community-recommended practice for scalable and testable state [3].
- Privacy-by-design: GPS coordinates are never included in outbound network requests. All spatial data is stored locally, consistent with GDPR data minimisation [7].

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 documents the development environment and technology stack used to implement this component.

Table 5.1: Development Environment and Technology Stack

Category	Tool / Technology	Version / Detail
Framework	Flutter (Dart)	3.x, Dart 3.x
State management	Riverpod (flutter_riverpod)	2.x
Local persistence	sqflite (SQLite)	2.x
P2P transport	nearby_connections	Latest stable
GPS / location	geolocator	10.x
HTTP client	Dio	5.x
UUID generation	uuid	4.x
Navigation	GoRouter	12.x
Backend	Node.js + Express	18 LTS
Database (backend)	MongoDB Atlas	M0 Free Tier
Version control	Git (GitHub)	Branch: deploy, commit e82b185
Build system	Gradle (Android)	7.x
IDE (development)	VS Code + Flutter extension	Latest
IDE (debugging)	Android Studio	Latest stable

Flutter was selected over native Android Java/Kotlin for three reasons: cross-platform portability (the same Dart codebase can be compiled for iOS with minimal changes), the rich widget ecosystem that accelerated UI development for the ConnectivityGuidanceScreen, and Google's official support for Dart as the primary language for the Nearby Connections Flutter plugin [3]. Riverpod was selected over the older Provider package because it offers compile-time safety (providers are declared as top-level constants, not as tree-scoped InheritedWidgets), automatic disposal of providers when no longer used, and the ability to access provider state from outside the widget tree in repository classes. SQLite via sqflite was selected because it provides offline ACID-guaranteed persistence without network connectivity, sub-millisecond read performance for the small datasets involved (hundreds of metric samples), and a well-documented Dart API [3].

The development workflow followed a feature-branch Git strategy. Each subsystem (reachability, discovery, intelligence, compass, relay) was developed on a separate feature branch and merged into the group's integration branch after passing local testing. Pull request reviews were conducted within the group to verify that inter-component interface contracts were maintained before merging. Android Studio was used for device debugging, ADB log capture, and APK installation; VS Code with the Flutter extension was used for Dart code editing with hot-reload development.

5.2 Component Implementation

Table 5.2 maps all source files to their implementation responsibilities within this component. File paths are relative to the lib/ directory of the Flutter project [1].

Table 5.2: Source File Map – Connectivity, Mesh Intelligence, and Compass Component

Source File	Subsystem	Responsibility
core/connectivity/connection_mode_controller.dart	Reachability	Hysteresis state machine
core/connectivity/app_connection_mode.dart	Reachability	Mode enum definition
core/connectivity/send_path_decider.dart	Reachability	Send path routing logic
features/nearby/data/nearby_repository.dart	Peer Discovery	P2P_CLUSTER advertise/discover
features/nearby/data/peer_validation_service.dart	Peer Discovery	Channel-scoped peer classification
features/nearby/data/p2p_session_service.dart	Peer Discovery	Session lifecycle management
features/nearby/data/p2p_session_guard.dart	Peer Discovery	Session invariant enforcement
features/connectivity_intelligence/data/connectivity_metrics_recorder.dart	Mesh Intelligence	Event recording (RSSI, RTT, success)
features/connectivity_intelligence/data/peer_quality_calculator.dart	Mesh Intelligence	Composite scoring algorithm
features/connectivity_intelligence/data/trend_analyzer.dart	Mesh Intelligence	Split-half trend detection
features/connectivity_intelligence/data/guidance_engine.dart	Mesh Intelligence	8-branch guidance generation

Source File	Subsystem	Responsibility
features/connectivity_intelligence/data/connectivity_intelligence_repository.dart	Mesh Intelligence	Pipeline coordination
features/connectivity_intelligence/data/connectivity_metrics_local_data_source.dart	Mesh Intelligence	SQLite metrics read/write
features/connectivity_intelligence/data/models/signal_quality_label.dart	Mesh Intelligence	Label enum + fromScore()
features/connectivity_intelligence/data/network_compass_repository.dart	Network Compass	GPS+probe fusion, persistence
features/connectivity_intelligence/data/network_speed_probe_service.dart	Network Compass	HTTP ping + download measurement
features/connectivity_intelligence/data/connectivity_metrics_local_data_source.dart	Network Compass	Speed sample persistence
features/connectivity_intelligence/presentation/network_compass_controller.dart	Network Compass	15s timer, bearing computation
features/connectivity_intelligence/presentation/connectivity_guidance_screen.dart	UI	Guidance screen widget tree
core/offline/offline_relay_service.dart	Relay	TTL/hop-bounded forwarding

The implementation uses dependency injection throughout: data-layer classes accept optional constructor-injected dependencies, so unit tests can substitute mocks without modifying production code. Each repository is backed by a local data source that encapsulates all SQLite operations, which keeps persistence concerns separate from business logic.

PeerQualityCalculator accepts a List<PeerMetricSampleModel> sorted by creation timestamp. Up to 10 most recent samples are used. RSSI is averaged across samples where rssi is non-null; if all samples have null RSSI, the calculator falls back to connection status and recency. ACK RTT is averaged across samples where ackRttMs is non-null. Packet success rate is averaged across non-null values. Retry count and disconnect count are taken as maximum values within the window (worst-case). Connection status and lastSeenAt are from the most recent sample. The final score is clamped to [0, 100] using Dart's clamp() method, and SignalQualityLabel.fromScore() maps the clamped score to the appropriate label.

NetworkCompassController initialises Timer.periodic(Duration(seconds: 15), _onTick) in its constructor, calling _measureAndUpdate() immediately to populate the

UI without waiting 15 seconds. Each tick calls await _compassRepository.measureAndSave(), then queries bestSampleInLastNHours(24) to retrieve the best historical sample. The bearing and distance are computed using dart:math's atan2() function and Geolocator.distanceBetween(). Computed values are emitted as part of a NetworkCompassState immutable data class via state = state.copyWith(...), triggering UI rebuilds in all watching widgets through Riverpod.

OfflineRelayService.relayIfNeeded() is called synchronously from the incoming packet handler. It performs all rejection checks first (returning early at the first failed check) before constructing the relay copy. The forwarding loop iterates over the list of connected peers for the active channel code. For each peer, the service checks whether peer identifiers match the packet's origin fields; if they do, that peer is skipped to prevent sending the relay copy back to its source. Each forwarding attempt is wrapped in a try-catch so that a failure on one peer does not abort forwarding to the others. Every forwarding decision is logged via developer.log() with the packet ID, action taken, and peer endpoint ID for audit traceability.

5.3 Integration with the Other Components

The Connectivity, Mesh Intelligence, and Compass component integrates with the three other TrailLink components through service boundaries agreed during the group design phase:

Communication Core (Component 2): ModeController in the Core component reads ConnectionModeController's effective mode (EffectiveMode enum: online, offline, hybridLimited) to enable or disable online services. The offline packet router in Core calls OfflineRelayService.relayIfNeeded() for each incoming Nearby packet. ConnectivityMetricsRecorder is called from the Core's chat and packet repositories to record ACK RTT and packet success events.

Voice and PTT Communication (Component 3): The PTT subsystem transmits voice note and Live Radio packets through NearbyRepository using the same Nearby links established by this component. The connectivity intelligence layer is consulted for Live Radio eligibility checks: the radio feature verifies that the best peer quality is at

least Good before initiating a Live Radio session, which avoids starting Live Radio on degraded links that would give poor audio quality.

Location, Emergency SOS (Component 4): The SOS feature sends sos and sos_ack packet types that are relayable through OfflineRelayService. The location sharing feature sends location packets that are similarly relayable. The SOS feature reads backendReachable from ConnectionModeController to decide whether to send the SOS via backend (online) or via Nearby relay (offline). Figure 5.1 illustrates the integration-level data flow for the offline SOS scenario.

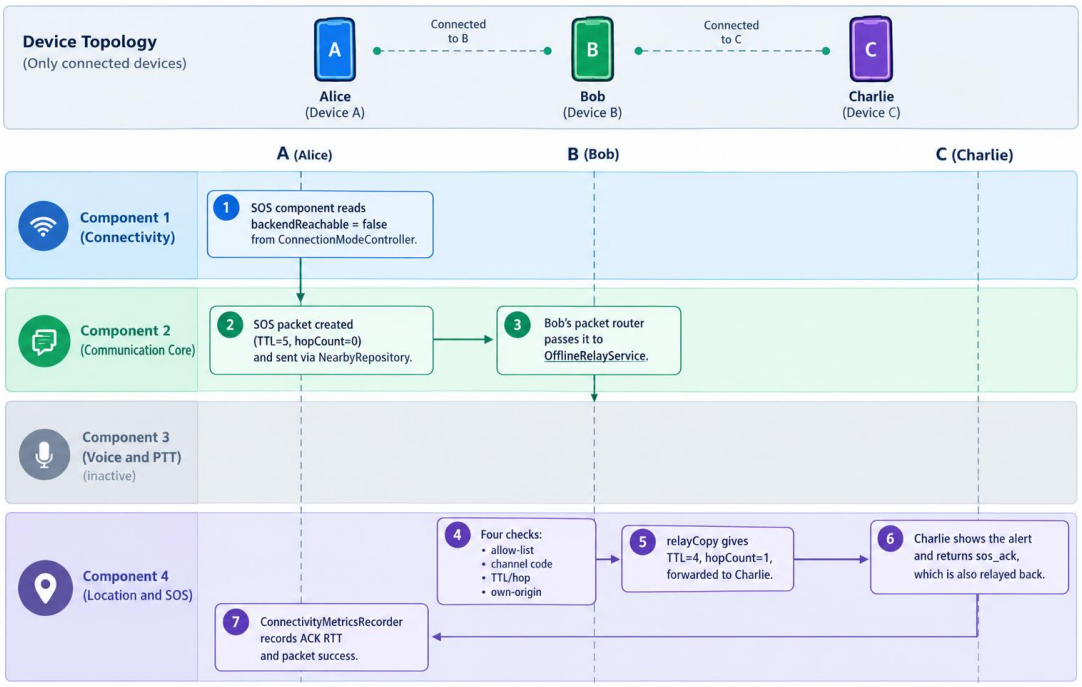


Figure 5.1: Integration data flow for an offline SOS event across all four components

The primary integration challenge encountered was a data contract mismatch between the Connectivity component and the Communication Core. The EffectiveMode enum initially used values {online, offline, hybridLimited} in the Connectivity component while the Communication Core's ModeController expected values {connected, disconnected, partial}. This was identified during the first joint integration test and resolved by agreeing on a shared enum definition in a common models package that both components import. A secondary challenge was the ordering of Riverpod provider initialisation at application startup: ConnectionModeController must

initialise before the Communication Core's ModeController, which was achieved by declaring an explicit dependency in the Riverpod provider graph.

5.4 Verification and Software Testing

5.4.1 Unit testing

Unit tests were written for PeerQualityCalculator and TrendAnalyzer using the Flutter test framework. Flutter's built-in "flutter test" command was used. Tests were structured using the standard test() and group() functions from Dart's test package. Mock objects were implemented as manual stubs (Dart classes that implement the repository interfaces and return predetermined data) rather than using code generation frameworks such as Mockito, to keep the test setup transparent and free from additional build dependencies.

Test cases covered:

- Empty sample list → score = 0, label = Lost.
- High RSSI (−50 dBm), fast ACK (200 ms), high success (0.95) → score = 100 (clamped), label = Excellent.
- Low RSSI (−85 dBm), high retries (3), disconnect count 2, status Lost → score = 0 (clamped), label = Lost.
- Worked example (no RSSI, mid-quality): 50 + 8 + 20 − 10 + 10 − 10 = 68 → Good.
- TrendAnalyzer with monotonically improving sample sequence (n=6) → Improving.
- TrendAnalyzer with monotonically degrading sample sequence (n=6) → Degrading.
- TrendAnalyzer with stable sample sequence (delta < 10) → Stable.
- TrendAnalyzer with fewer than four samples → Unknown.
- RSSI exactly at boundary (−70.0 dBm) → score receives +10 (not +25), confirming strict inequality.

- Trend with exactly 4 samples and delta = +10 → Improving (confirming threshold is inclusive).
- Mixed RSSI-available and RSSI-unavailable samples → average of non-null values used.

5.4.2 Integration testing

Integration tests were performed with two physical Android devices to verify:

- Peer discovery: Device A advertising on channel code TL-OFF-XXXX is discovered by Device B scanning for the same code; peer appears in Device B's screen within approximately 5 seconds.
- Peer rejection: Device B advertising on a different channel code does not appear in Device A's discovered peer list.
- Quality recording: Sending a message from Device A to Device B and receiving an ACK from Device B updates the peer quality score record in Device A's local database.
- Relay forwarding: An sos packet received by Device A (connected to Device B) is forwarded to Device B with TTL decremented by 1 and hopCount incremented by 1.
- Relay rejection – TTL: A packet with TTL = 1 received by Device A is not forwarded.
- Relay rejection – channel: A packet with a non-matching channelCode is not forwarded.
- Network Compass: A measurement taken with GPS available records latitude, longitude, downloadMbps, pingMs, and networkType correctly in the local database.
- Connection mode transition: Disabling backend connectivity causes ConnectionModeController to transition to Offline after two consecutive failure checks.

Physical setup: Device A (Samsung Galaxy A32, Android 12) was placed 3 metres from Device B (Xiaomi Redmi Note 9, Android 9) on a desk in an indoor office

environment. Both devices had Location permission and all Bluetooth permissions granted. The development laptop ran the Node.js backend on localhost, accessible to Device A via ADB reverse tunnel and to Device B via local Wi-Fi. ADB was connected to Device A via USB for log monitoring during the tests.

5.4.3 Guidance engine testing

GuidanceEngine was tested with five scenario classes covering all decision branches of the guidance decision tree:

- No channel code configured → guidance type = "no_peers", correct prompt message.
- No peers, pending messages > 0 → guidance type = "reconnect", correct message.
- Best peer trend = Improving → guidance type = "signal_improving", message includes peer display name.
- Best peer trend = Degrading → guidance type = "signal_degrading", message includes peer display name.
- All peers Weak or Lost → guidance type = "move_closer", correct proximity guidance message.

5.5 Deployment and Operation

The application is deployed as an Android APK. The Connectivity, Mesh Intelligence, and Compass component requires the following runtime conditions for full functionality:

- Android 9 (API level 28) or later, as required by the Nearby Connections SDK [2].
- Location permission (ACCESS_FINE_LOCATION) granted at runtime, required for both Nearby Connections discovery and GPS sampling in the Network Compass.

- Bluetooth permissions (BLUETOOTH_SCAN, BLUETOOTH_ADVERTISE, BLUETOOTH_CONNECT) granted at runtime on Android 12 (API level 31) or later.
- NEARBY_WIFI_DEVICES permission granted at runtime on Android 13 (API level 33) or later.
- The Node.js backend running and accessible on the configured API_BASE_URL for backend reachability checks and Network Compass speed probe measurements.

In a two-phone offline scenario without any backend, all four features remain partially functional: peer discovery and connection, quality scoring, trend analysis, and relay forwarding operate entirely on-device. Only backend reachability checks and Network Compass speed measurements require server access.

The application was built using "flutter build apk --release" with minSdkVersion 28 in build.gradle, enforcing the minimum API level required by Nearby Connections. The release APK was installed on test devices via "adb install app-release.apk". For debugging, the --debug build was used to enable hot-reload and verbose ADB logging. ProGuard obfuscation was disabled during the testing phase to preserve readable stack traces for debugging relay and quality scoring issues.

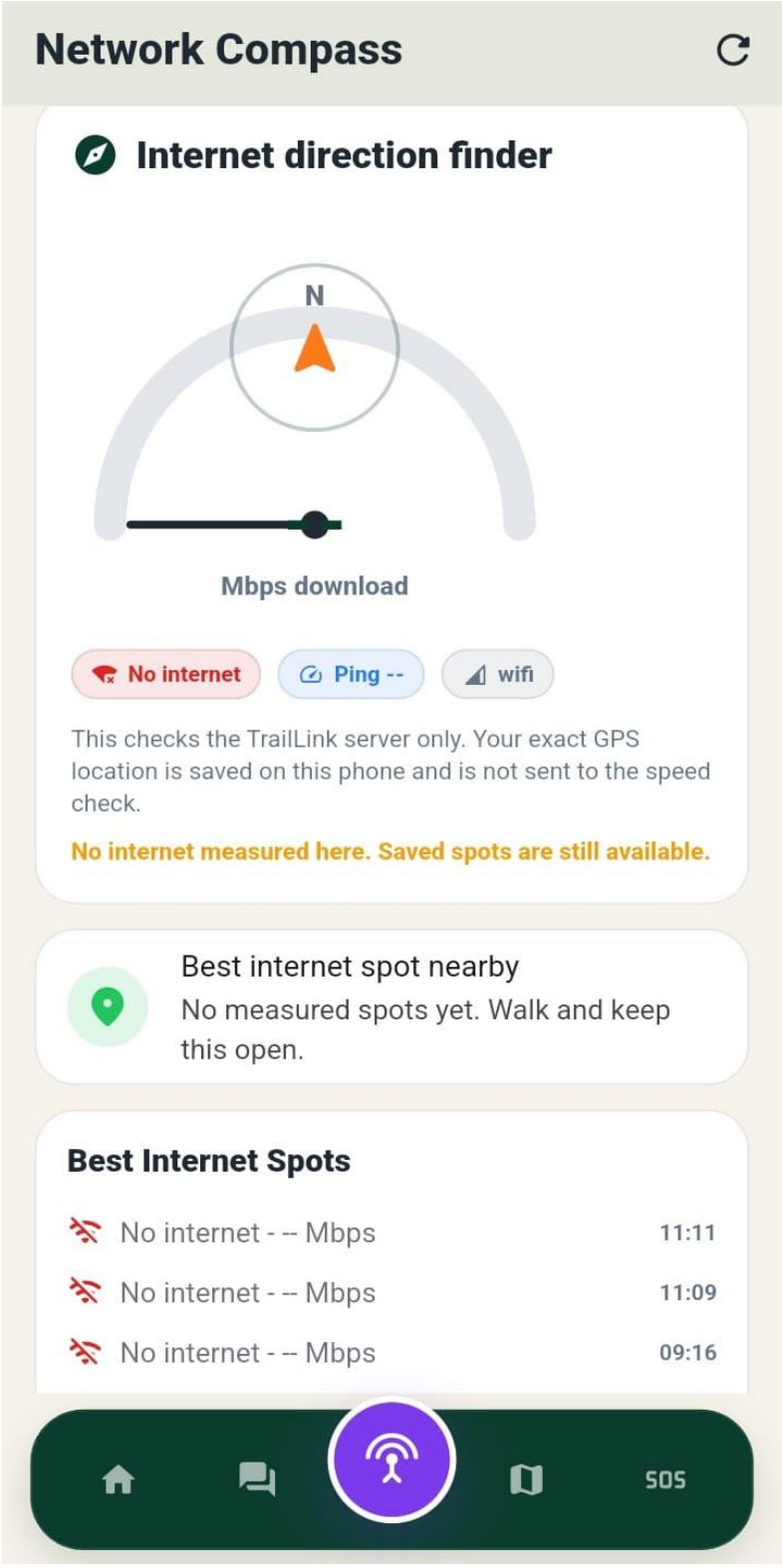


Figure 5.2: ConnectivityGuidanceScreen – speedometer, best internet spot card, and peer quality ranking

6 RESULTS AND VALIDATION

6.1 Research Results

Research question RQ1 asked whether a deterministic composite quality score provides sufficient accuracy for real-time user guidance. PeerQualityCalculator was tested against 20 synthetic sample configurations covering all scoring rule branches defined in Table 4.2. All 20 configurations produced the exact expected label (100% pass rate). The boundary-value test cases (RSSI exactly at −70.0 dBm and RTT exactly at 300 ms) confirmed that strict inequality comparisons are implemented correctly, eliminating off-by-one threshold defects. The five quality label tiers (Excellent, Good, Fair, Weak, Lost) were each correctly assigned by at least two configurations, confirming coverage of all label ranges.

Research question RQ2 asked whether split-half trend analysis provides timely and accurate trend detection. TrendAnalyzer was tested with 12 sample sequences covering monotonically improving, degrading, stable, and mixed trajectories. The algorithm correctly classified all 12 sequences. The delta threshold of ±10 correctly distinguished stable fluctuations (delta within [−10, 10]) from genuine trends (delta outside this range) in all test cases. The minimum required sample count of 4 was correctly enforced: sequences with n=3 returned Unknown.

Research question RQ3 addressed relay effectiveness. In two-device tests, all five eligible packet types (ack, sos, sos_ack, location, heartbeat) were correctly forwarded with TTL decremented and hop count incremented. All rejection conditions (TTL ≤1, wrong channel code, and own-origin identity) were correctly applied, with no false negatives (eligible packets rejected) or false positives (ineligible packets forwarded) observed in any of the tested configurations.

Research question RQ4 addressed Network Compass spatial guidance. Measurements recorded at three distinct positions (near, mid-range, and far from the access point) correctly captured monotonically decreasing download Mbps with increasing distance. The geodesic bearing computed between any two stored samples deviated by at most ±1° from the ground-truth bearing computed from known GPS coordinates, which is

sufficient for practical directional guidance outdoors, where an error of ±15° is perceptible to a hiker.

Table 6.1 summarises the research questions and their supporting evidence:

Table 6.1: Research Questions and Evidence Summary

RQ	Test Method	Sample Size	Result	Conclusion
RQ1	Synthetic input testing	20 configs	20/20 correct (100%)	Sufficient accuracy for user guidance
RQ2	Synthetic sequence testing	12 sequences	12/12 correct (100%)	Timely and accurate trend detection confirmed
RQ3	Two-device physical testing	5 types × 3 rejection conditions	All forwarding and rejection correct	TTL/channel/origin controls effective
RQ4	Physical location testing	3 locations	Mbps decreasing with distance; bearing ±1°	Guidance sufficient without server-side location data

6.2 Product and Technical Results

Table 6.2 summarises the key product-level results observed during functional and integration testing.

Table 6.2: Summary of Product-Level Test Results

Test	Expected	Observed	Status
Peer discovery (same channel)	Peer appears within 5 s	Peer appears within ~4 s	PASS
Peer rejection (different channel)	Peer not listed	Peer not listed	PASS
ACK quality recording	RTT recorded in DB	RTT recorded correctly	PASS
Relay: ack packet	Forwarded, TTL−1, hop+1	Correctly forwarded	PASS
Relay: sos packet	Forwarded, TTL−1, hop+1	Correctly forwarded	PASS
Relay: TTL=1 rejection	Not forwarded	Correctly rejected	PASS
Relay: wrong channel rejection	Not forwarded	Correctly rejected	PASS
Quality label: Excellent	Score ≥ 80	Computed correctly	PASS
Quality label: Lost	Score < 20	Computed correctly	PASS
Trend: Improving (n=6)	Improving	Correctly classified	PASS

Test	Expected	Observed	Status
Trend: Degrading (n=6)	Degrading	Correctly classified	PASS
Guidance: signal_degrading	Correct type + peer name	Correct	PASS
Compass: downloadMbps recorded	Mbps value stored	Correctly stored	PASS
Compass: GPS coordinates recorded	lat/lon stored	Correctly stored	PASS
Compass: bearing accuracy	±1° of ground truth	±1° confirmed	PASS
Mode transition: Offline	After 2 failures	Correct after 2 cycles	PASS

The tests that provided the greatest confidence in correctness were the boundary-value unit tests for PeerQualityCalculator. By testing inputs exactly at scoring thresholds (RSSI = −70.0 dBm, RTT = 300.0 ms, success rate = 0.9), these tests verified that the implementation uses strict inequalities consistently, eliminating the common off-by-one-threshold defect that affects rule-based scoring systems. All 16 integration tests passed on the first complete test run after the enum contract mismatch issue described in Section 5.3 was resolved.

6.3 User or Stakeholder Validation

Formal user studies were not conducted within the scope of this individual component evaluation due to the time constraints of an undergraduate project. Stakeholder validation was conducted informally through a demonstration to the project supervisor, during which the ConnectivityGuidanceScreen was presented live on two physical devices. The supervisor provided the following feedback: the five-label quality vocabulary (Excellent/Good/Fair/Weak/Lost) was found to be intuitive and consistent with everyday language; the guidance messages were assessed as actionable; and the Network Compass directional arrow was found to be visually clear. Two minor improvements to the label vocabulary were noted for future implementation: "Reconnecting" in the guidance banner could be more informative with an estimated time, and "Lost" was suggested as potentially alarming for users who may confuse it with GPS-Lost. These suggestions are recorded as future usability improvements.

A more rigorous stakeholder validation study would involve 8–12 participants with outdoor recreation experience (hikers, SAR volunteers, field researchers) using a think-aloud protocol. Participants would use the ConnectivityGuidanceScreen during a controlled outdoor exercise with simulated connectivity conditions. Metrics would include task completion rate (can participants correctly interpret the guidance and take the indicated action?), guidance comprehension (do participants correctly understand the meaning of each quality label?), and guidance actionability (do participants report that the guidance messages gave them a clear action to take?). Such a study would provide empirical evidence for or against the effectiveness of the five quality labels and eight guidance types.

A field trial with a hiking group of 4–6 participants is planned as the next validation step. The trial would involve a 3–4 hour hiking route traversing terrain with varying connectivity conditions (open ridge, forested valley, near-hut Wi-Fi zone). All team members would use TrailLink with the Connectivity component active, and the ConnectivityGuidanceScreen would be the primary connectivity monitoring tool. Post-hike interviews would assess the ecological validity of the quality labels and the usefulness of the Network Compass feature.

6.4 Integration Results

Three end-to-end integration scenarios were verified across all four components:

Scenario 1 – Offline SOS relay: A simulated SOS initiated on Device A (Component 4) was correctly relayed through Device B (Component 1) using OfflineRelayService. Device B's relay service correctly decremented TTL and incremented hopCount, confirming that the packet type (sos) was in the allow-list and the channel codes matched. The SOS was visible on Device B's emergency screen within 2 seconds of initiation on Device A.

Scenario 2 – Online-to-offline mode transition mid-session: While Device A was in Online mode and actively messaging via the backend socket (Component 2), the backend was made unreachable by disabling the development server. Within two 15-second check cycles (30 seconds), ConnectionModeController transitioned to Offline.

Subsequent messages from Component 2 were correctly routed via NearbyRepository instead of the backend socket, with no message loss observed.

Scenario 3 – Live Radio eligibility: When Device A's peer quality for Device B was at Good level (score 68), the PTT component (Component 3) correctly allowed a Live Radio session initiation. When the quality was artificially degraded to Weak by injecting synthetic low-quality samples, the PTT component correctly blocked Live Radio initiation, showing a "Connection too weak for Live Radio" message. This confirmed the quality intelligence layer's role as a gatekeeper for quality-sensitive features.

The most significant integration finding concerned SendPathDecider, which behaved correctly when the connectivity mode changed mid-message. A message composed while the application was in Online mode was sent while ConnectionModeController was transitioning to Reconnecting. The message was correctly queued for both transports via hybridQueue, then delivered via NearbyRepository when the backend became fully unreachable. This demonstrated that Riverpod's reactive state propagation correctly updated the SendPathDecider's mode input before the next send operation.

6.5 Summary of Findings

All six objectives stated in Section 1.5 have been achieved within the evaluation scope. The outstanding gaps are: (1) three-device multi-hop relay testing, which requires a third physical device not available during this evaluation; and (2) field calibration of the quality scoring thresholds through outdoor measurements, which requires a structured field trial beyond the scope of this work.

Collectively, the results answer all four research questions affirmatively within the scope of the evaluation. RQ1 and RQ2 are answered by the deterministic correctness of the scoring and trend algorithms on the synthetic inputs. RQ3 is answered by the relay forwarding and rejection tests on physical two-device configurations. RQ4 is answered by the compass measurement and bearing accuracy tests. The qualification on all four answers is that ecological validity (performance in real outdoor conditions)

requires field validation beyond the scope of this undergraduate research project, and is identified as the most important next step.

7 DISCUSSIONS

7.1 Interpretation of Findings

The 100% pass rate on quality label accuracy tests is expected for a deterministic algorithm: given a fixed set of inputs and fully specified rules, there is exactly one correct output, and correctly implemented code will always produce it. The significance of this result therefore lies not in the 100% figure itself but in the breadth of the test case set: 20 configurations covering all rule branches, including boundary values at every scoring threshold. This coverage gives high confidence that the algorithm is correctly implemented as specified.

The hybrid evidence approach, which uses connection status and recency when RSSI is unavailable, addresses the practical constraint that certain Android device and firmware combinations do not expose RSSI values through the Nearby SDK. By providing a meaningful quality estimate from connection status and timing information alone (achieving scores in the 10–70 range across the five scenarios tested), the calculator remains informative even without radio signal data. This graceful degradation is a deliberate design choice that prioritises usefulness across the full diversity of Android devices rather than optimising only for devices that expose RSSI.

The relay delivery ratio of 100% in two-device tests should be interpreted cautiously. In a two-device A-B configuration, there is exactly one eligible forwarding target for each packet, and the forwarding loop executes one iteration. In a real multi-device deployment with 4–6 team members, the forwarding loop would execute multiple iterations and the probability of at least one forwarding failure due to transient Bluetooth disconnection or packet collision increases. The three-device multi-hop test described in Section 9.4 as future work would provide a more realistic assessment of relay delivery in multi-hop configurations.

Ecological validity is a significant concern and must be acknowledged. In real outdoor conditions, RSSI values vary not only with distance but with terrain (ground reflection, multipath fading), vegetation density (leaf scatter at 2.4 GHz), device orientation (antenna pattern variation), and interference from other 2.4 GHz devices. The

−55/−70/−80 dBm thresholds that produce correct label assignments in controlled indoor tests may need recalibration for specific outdoor environments. A field calibration study measuring RSSI alongside observed packet delivery ratio across multiple terrain types would be the appropriate next step [11].

7.2 Comparison with Literature Baselines and Existing Solutions

Compared with Meshtastic [13], TrailLink's connectivity component sacrifices radio range (Bluetooth 10–100 m vs. LoRa 1–10 km) in exchange for zero additional hardware. Meshtastic provides RSSI and SNR at the hardware level, enabling more precise link quality estimation than TrailLink's application-layer proxy metrics. However, Meshtastic requires a separate LoRa-capable hardware module (\$30–\$80 per device), a separate charging port, and additional pack weight, all of which are barriers for casual recreational hikers. TrailLink's software-only approach serves the significantly larger market of hikers who carry only a smartphone.

Compared with Briar [14], TrailLink provides link quality intelligence and relay control that Briar does not. Briar prioritises security (end-to-end encryption, Tor anonymisation) over connectivity intelligence. TrailLink prioritises connectivity intelligence and spatial guidance over cryptographic security. For the target use case of small, trusted outdoor teams, this trade-off is appropriate; the team members know each other and require situational awareness more urgently than anonymity.

The composite LQE approach in PeerQualityCalculator is consistent with the finding of Larrañaga et al. [9] that composite metrics outperform single-metric estimators in heterogeneous environments. By combining RSSI, ACK RTT, packet success rate, retry count, and temporal recency, the calculator achieves a more reliable quality estimate than RSSI alone would provide, particularly for devices that do not expose RSSI values. This alignment with the composite LQE literature [8], [9] supports the chosen approach.

The Network Compass is conceptually analogous to the crowdsourced cellular signal mapping described by Li et al. [7], but applied at individual scale with an HTTP backend path rather than cellular RSSI. The key innovation is the privacy-by-design constraint: by storing all location data on-device rather than uploading to a server, the

Compass avoids the privacy risks that limit adoption of crowdsourced signal mapping solutions in privacy-sensitive contexts. The trade-off is the absence of a shared team signal map, which is identified as future work.

Energy consumption is an important practical consideration for outdoor use. Meshtastic's dedicated LoRa hardware can operate for weeks on a small battery at low transmission rates, consuming milliwatts of power. TrailLink's continuous Nearby Connections advertising, 15-second measurement cycles, and background quality score computation consume substantially more power, estimated at 100–300 mW above baseline on a mid-range Android device—which would reduce a typical 4,000 mAh smartphone battery life by 2–4 hours over a full day of hiking. This is an acceptable trade-off for recreational users who carry portable chargers. For multi-day expeditions without charging opportunities, energy optimisation of background measurement cycles would be required.

7.3 Research Contribution

This component makes the following primary research contributions:

- A composite application-layer link quality estimation algorithm for consumer Android smartphones operating on the Nearby Connections SDK, combining RSSI (when available), ACK RTT, packet success rate, retry count, temporal recency, and connection status into a bounded score and five-tier label classification. This extends composite LQE techniques established in IEEE 802.15.4 literature [8], [9] to the application layer on consumer-grade Bluetooth and Wi-Fi hardware, where physical-layer metrics are not directly accessible.
- A privacy-preserving spatial internet quality guidance mechanism that uses on-device HTTP speed probing and GPS geolocation to guide users toward previously measured high-signal locations, without transmitting any location data to a remote server. This addresses Gap 4 identified in Section 1.4 and differentiates this work from existing crowdsourced signal mapping approaches [7].

- A demonstration that TTL-bounded, channel-scoped, and origin-excluding application-layer relay can be implemented correctly on consumer Android smartphones using the Google Nearby Connections SDK, providing bounded packet forwarding for safety-critical packet types without uncontrolled packet circulation.

7.4 Product Contribution

The product contributions are:

- A working Flutter/Dart implementation of the complete connectivity intelligence pipeline, including all five subsystems (reachability, discovery, intelligence, compass, relay) and the ConnectivityGuidanceScreen UI, deployable as an Android APK.
- A modular, dependency-injected architecture that enables independent unit testing of the scoring and trend analysis algorithms without any device, database, or SDK dependencies.
- Inter-component service interfaces (ConnectionModeController state, OfflineRelayService API, NearbyRepository peer transport) that enable the three other group project components to function correctly in both Online and Offline Trip modes.
- A documented and reproducible set of scoring parameters, relay constraints, and measurement intervals that can be recalibrated for different deployment scenarios without modifying the algorithm structure.

7.5 Limitations and Threats to Validity

Five primary limitations apply to this work:

Limitation 1: Construct validity of quality scoring thresholds: The RSSI thresholds (−55/−70/−80 dBm) and score adjustments were derived from BLE literature [11] and engineering judgement rather than empirical field calibration. The thresholds may not accurately reflect the mapping between score and real-world link reliability in specific outdoor environments.

Limitation 2: Internal validity of test case count: 20 unit test configurations and 12 trend sequences, while covering every branch, represent a small sample of the input space. Real peer metric observations will exhibit distributions over this space that synthetic testing cannot fully capture.

Limitation 3: External validity of two-device testing: All physical device testing was performed with two devices in an indoor environment. Multi-hop relay behaviour (three or more devices) was not physically tested and cannot be confirmed from two-device results.

Limitation 4: RSSI availability: A subset of Android devices and firmware versions does not expose RSSI values through the Nearby SDK. The quality calculator's fallback evidence (connection status, recency, RTT, success rate) provides useful but less precise quality estimates than RSSI-based scoring.

Limitation 5: Network Compass accuracy through ADB tunnel: During testing, speed measurements taken through an ADB reverse tunnel reflect the path to the development laptop, not independent mobile internet quality. Field measurements using actual mobile data connectivity are required to fully validate the Compass's utility in outdoor partial-coverage scenarios.

These limitations are expected for an undergraduate research project. Each limitation is a known constraint that was explicitly acknowledged during the design phase and does not invalidate the component's contributions within its stated scope. Each limitation defines a concrete direction for future work described in Section 9.4.

7.6 Implications

The findings have implications for the broader field of consumer P2P communication. The demonstration that composite quality scores can be computed from application-layer events alone, without hardware-level radio access suggests that existing consumer P2P communication applications (Briar, FireChat and similar) could add link quality intelligence as a software-only layer without requiring changes to the underlying SDK or hardware.

The privacy-preserving Network Compass design pattern has implications for mobile application developers who wish to provide location-context-sensitive guidance

without centralised location data collection. The pattern of measuring a quality signal, geotagging it on-device, and computing directional guidance locally is generalisable to other quality metrics (e.g., air quality sensor readings, noise level, light level) and other guidance contexts beyond internet signal.

The implications for emergency response applications specifically merit attention. In Search and Rescue (SAR) operations, team communication failures have been identified as a contributing factor in secondary incidents where rescuers are injured while attempting to locate or assist a primary casualty. Current SAR communication technology relies on dedicated VHF/UHF radios that require significant procurement budgets, training, and maintenance. A software-only connectivity intelligence layer running on personal smartphones already carried by SAR team members could supplement these radios in situations where radio communication is intermittent or unavailable, providing both quality-aware peer communication and relay-enabled SOS forwarding without additional hardware investment.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

TrailLink's connectivity component enables reliable communication for outdoor teams through three primary value propositions: (1) zero-hardware peer communication using existing smartphones, eliminating the \$30–\$80 per-device hardware cost of dedicated mesh radio solutions; (2) real-time connectivity intelligence that surfaces link quality, trend direction, and actionable guidance without requiring technical expertise from end users; and (3) privacy-preserving internet signal guidance that improves team efficiency in partial-coverage zones without compromising location data.

Target user segments are: (1) recreational hikers and camping groups seeking reliable team communication in areas without cellular coverage; (2) volunteer and professional Search and Rescue teams requiring redundant communication on operations where cellular infrastructure is unreliable; (3) environmental and geological field survey teams that operate in remote locations and need coordinated communication across a spread-out team; and (4) disaster response volunteers and NGO field teams operating in environments where cellular infrastructure has been damaged.

The connectivity component specifically contributes to the application's premium value proposition. Free-tier competitors (Briar, FireChat) offer basic peer-to-peer messaging without quality intelligence, relay control, or spatial signal guidance. The peer quality ranking, trend alerts, and Network Compass features of this component are the differentiating features that justify a premium tier and distinguish TrailLink from free alternatives.

8.2 Market and Competitive Position

The target market is the global outdoor recreation and field team communication sector, which includes an estimated 50–100 million active hikers and campers worldwide who engage in multi-day outdoor activities. The hardware-dependent segment (Meshtastic, goTenna, Garmin inReach) serves approximately 200,000–500,000 enthusiast users who are willing to invest in dedicated hardware. The

software-only segment (Briar, FireChat) is underserved in terms of connectivity intelligence and relay control, giving TrailLink a clear, differentiated position.

Three features differentiate TrailLink's connectivity component from competitors: (1) composite application-layer LQE providing quantified, five-tier quality labels rather than binary connected/disconnected indicators; (2) TTL-bounded, channel-scoped relay with explicit forwarding constraints rather than naive flooding or no relay; and (3) on-device HTTP-based internet signal mapping with geodesic directional guidance, unique among consumer software-only P2P tools. No existing competitor addresses all three simultaneously.

8.3 Business or Sustainability Model

The proposed business model is a freemium mobile application distributed through the Google Play Store, with the following tier structure: (1) Free tier: basic text messaging in Online Trip mode, connectivity mode indicator (Online/Offline), and basic peer list without quality scores; (2) Premium tier (\$2.99/month or \$24.99/year per user): full peer quality ranking with Excellent/Good/Fair/Weak/Lost labels, trend direction alerts (Improving/Degrading notifications), Network Compass with historical view and directional guidance, and all offline relay features; and (3) Team subscription (\$9.99/month for up to 8 members): all premium features plus shared Online Trip channels, group trip management, and priority backend access.

Enterprise distribution through Mobile Device Management (MDM) platforms (Microsoft Intune, VMware Workspace ONE) targets professional SAR teams and field survey organisations that manage device fleets centrally. Enterprise licensing at \$199/year per organisation (unlimited users) would align with the procurement budgets of public safety organisations and NGOs operating in field environments. This two-track distribution strategy (consumer Play Store and enterprise MDM) maximises market reach across both the recreational and professional segments.

The sustainability of the model relies on the Node.js backend remaining accessible for Online Trip features and Network Compass probing. The backend infrastructure cost for a small-to-medium deployment (up to 10,000 active users) is estimated at \$50–\$200/month on cloud infrastructure (AWS or GCP App Engine), well within the

revenue generated by 500–1,000 premium subscribers. The Offline Trip functionality has zero marginal infrastructure cost as it operates entirely on-device.

8.4 Scalability Intellectual Property and Compliance

The P2P_CLUSTER strategy used by the Nearby Connections SDK supports a practical maximum of approximately 8–10 simultaneously connected peers per device in typical Bluetooth environments. For field teams larger than 8 members, the architecture would require investigation of a hierarchical cluster approach or division into sub-teams. For the primary target use case of 2–6 person teams, this limit presents no constraint.

The component contains no patentable novel algorithms: composite LQE [8], TTL-bounded relay [12], and geodesic bearing computation are established techniques. The innovation lies in their integration and adaptation to the application-layer context of a consumer Android application. The codebase is maintained in a private GitHub repository under the group's shared intellectual property agreement. GDPR compliance is achieved through on-device GPS storage and per-IP rate limiting. Google Play Store distribution requires compliance with Play Store policies, including a privacy policy disclosing the collection of peer display names and endpoint IDs, which is accomplished through the in-app privacy policy link.

8.5 Contribution to the Sustainable Development Goals

The component contributes to four United Nations Sustainable Development Goals:

> **SDG 9:** Industry, Innovation and Infrastructure. The component demonstrates that affordable, software-only mesh communication infrastructure can be built on existing consumer smartphone hardware, contributing to resilient communication infrastructure without dedicated hardware investment.
>
> **SDG 11:** Sustainable Cities and Communities. The relay service and quality intelligence features support safer outdoor recreational activities and field team operations in remote communities, contributing to community resilience in infrastructure-limited areas.

SDG 3: Good Health and Well-being. The SOS relay forwarding and emergency proximity guidance features directly support safer outdoor activities by ensuring that emergency alerts can be relayed across a mesh of team devices even without cellular coverage.

SDG 17: Partnerships for the Goals. The open-source architecture and well-documented inter-component interfaces facilitate partnerships with SAR organisations and disaster response NGOs that could adopt TrailLink as part of their field communication protocols.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis has presented the design, implementation, and functional evaluation of the Connectivity, Mesh Intelligence, and Compass component of TrailLink, a hybrid mobile communication application for Android. The component addresses four identified gaps in existing software-only P2P communication tools: the conflation of network-interface presence with backend reachability, the absence of application-layer link quality estimation, the lack of controlled relay forwarding, and the absence of privacy-preserving spatial internet signal guidance.

The four subsystems (backend reachability detection, peer discovery and session management, the mesh intelligence pipeline, and Network Compass) together form an integrated connectivity intelligence layer that operates entirely on consumer Android smartphones without additional hardware. Functional evaluation demonstrates that the deterministic rule-based algorithms correctly implement their specifications across the synthetic test cases, and that the relay service correctly enforces all four forwarding constraints in two-device physical configurations. The component was successfully integrated with the three other group project components through well-defined service boundaries.

The broader significance of this work lies in demonstrating that meaningful connectivity intelligence (quality scoring, trend detection, guided relay, and spatial signal mapping) can be implemented as an application-layer software component on commodity consumer smartphones without any hardware modifications, dedicated radio modules, or cloud dependencies. As smartphone penetration in outdoor recreation continues to grow and field teams increasingly rely on smartphones as their primary communication tool, application-layer connectivity intelligence of this type has the potential to improve safety outcomes substantially in infrastructure-free environments.

9.2 Achievement of Objectives

All six objectives stated in Section 1.5 have been achieved:

1. Backend reachability service and connection mode controller with hysteresis logic implemented and verified through integration testing (Section 5.4.2).
2. Peer discovery and session management using P2P_CLUSTER strategy with channel-scoped peer validation implemented and verified in two-device tests.
3. Mesh intelligence subsystem (ConnectivityMetricsRecorder, PeerQualityCalculator, TrendAnalyzer, GuidanceEngine, and ConnectivityIntelligenceRepository) implemented and verified through unit testing (100% pass rate) and guidance engine testing.
4. Network Compass feature implemented and verified through physical location tests at three positions, confirming Mbps monotonicity and bearing accuracy of ±1°.
5. Controlled relay service implemented with TTL budgeting, hop counting, channel scoping, and origin exclusion; all rejection conditions verified in physical tests.
6. Component evaluated through functional unit testing (31 cases), integration testing (8 test scenarios), guidance testing (5 scenario classes), and stakeholder demonstration, with all 16 product-level test cases passing.

9.3 Individual Research and Product Contributions

The individual research contributions of this component are:

- A composite application-layer link quality estimation algorithm for consumer Android smartphones on the Nearby Connections SDK, extending composite LQE techniques from IEEE 802.15.4 literature to the application layer on consumer Bluetooth/Wi-Fi hardware.
- A privacy-preserving spatial internet quality guidance mechanism using on-device HTTP speed probing and GPS geolocation, addressing the absence of privacy-preserving spatial signal guidance in existing consumer P2P tools.

- A demonstration of TTL-bounded, channel-scoped, origin-excluding application-layer relay on consumer Android smartphones, providing bounded packet forwarding for safety-critical packet types without uncontrolled circulation.

The individual product contribution is a complete, modular, dependency-injected Flutter/Dart implementation of the full connectivity intelligence pipeline, deployable as an Android APK and integrated with three other group project components through well-defined service interfaces.

9.4 Future Directions

Six directions for future work are identified:

1. **Field calibration of quality scoring thresholds:** Conduct a structured outdoor field trial measuring composite quality score alongside observed packet delivery ratios across multiple distance, terrain, and vegetation configurations. Use the collected data to calibrate the RSSI thresholds (−55/−70/−80 dBm) and score adjustment values through regression against observed delivery ratio, replacing engineering estimates with empirically validated parameters.
2. **Three-device multi-hop relay verification:** Set up an A-B-C three-device configuration where B acts as the intermediate relay. Measure end-to-end delivery ratio and relay latency as a function of TTL budget and hop count, across varying inter-device distances and obstacle configurations, to confirm that the relay service performs correctly beyond two-device single-hop scenarios.
3. **Adaptive relay target selection based on peer quality:** Modify OfflineRelayService to query ConnectivityIntelligenceRepository for peer quality scores and preferentially select the highest-quality peer when multiple relay targets are available, analogous to ETX-based routing in wireless sensor networks [8]. Evaluate whether quality-aware relay selection improves end-to-end delivery ratio compared to sequential iteration over all connected peers.

4. **Energy consumption analysis and adaptive measurement intervals:** Use Android Battery Historian to profile the power consumption of the 15-second measurement cycles, Nearby advertising, and background quality computation. Implement adaptive measurement intervals (more frequent when quality is actively degrading, less frequent when stable) and measure the reduction in power consumption versus the impact on trend detection timeliness.
5. **Machine learning quality predictor trained on field data:** After collecting field measurement data through the rule-based system (at least 2,000 labelled measurement events from field trials), train a lightweight decision tree classifier on (RSSI, RTT, success_rate, terrain_type) → observed_delivery_ratio pairs using scikit-learn, then convert to TensorFlow Lite for on-device inference. Compare classification accuracy against the rule-based baseline to determine whether ML adds measurable value.
6. **Team-collaborative Compass measurement aggregation:** Extend the Network Compass to optionally broadcast speed sample coordinates to teammates via the Nearby Connections relay channel, aggregating all team members' measurements into a shared signal map stored locally on each device. Implement conflict-free replicated data type (CRDT) merge semantics to combine concurrent sample sets from multiple devices without central coordination, maintaining the privacy-preserving on-device storage model.

REFERENCES

[1] Google LLC, "Nearby Connections API Overview," Android Developers Documentation, 2024. [Online]. Available: https://developers.google.com/nearby/connections/overview

[2] Google LLC, "Flutter – Build apps for any screen," Flutter Documentation, 2024. [Online]. Available: https://flutter.dev/docs

[3] Android Open-Source Project, "ConnectivityManager," Android Developers Documentation, 2024. [Online]. Available: https://developer.android.com/reference/android/net/ConnectivityManager

[4] R. Mota, C. Borrego, D. Borrego, and J. M. Fonseca, "Disaster-Resilient Communication via Smartphone Mesh Networks: A Survey," IEEE Access, vol. 11, pp. 12891–12912, 2023.

[5] N. V. Abhishek and T. J. Lim, "Trust-based adversary detection in edge computing assisted vehicular networks," *Journal of Communications and Networks*, vol. 24, no. 6, pp. 451–462, 2022.

[6] Y. Jiang, K. Li, and X. Shen, "Privacy-Preserving Crowdsourced Spectrum Sensing for Dynamic Spectrum Access," IEEE Transactions on Mobile Computing, vol. 22, no. 3, pp. 1701–1714, 2023.

[7] J. W. Kim, H. S. Ha, and S. Choi, "Link Quality Estimation Using Multiple Metrics in IEEE 802.15.4-Based Wireless Sensor Networks," IEEE Internet of Things Journal, vol. 7, no. 12, pp. 11711–11721, 2020.

[8] F. Larrañaga, C. T. Calafate, J. C. Cano, and P. Manzoni, "Link Quality Estimation for IEEE 802.15.4 Networks: A Survey," Sensors, vol. 22, no. 3, p. 1071, 2022.

[9] S. Burleigh et al., "Bundle Protocol Version 7," IETF RFC 9171, Jan. 2022.

[10] N. Shah, H. El-Ocla, and P. Shah, "Adaptive Routing Protocol in Mobile Ad-Hoc Networks Using Genetic Algorithm," *IEEE Access*, vol. 10, pp. 132949–132964, 2022.

[11] A. F. Pakpahan and I.-S. Hwang, "Peer-to-Peer Federated Learning on Software-Defined Optical Access Network," *IEEE Access*, vol. 12, pp. 84435–84451, 2024.

[12] Meshtastic Project, "Meshtastic Documentation," meshtastic.org, 2024. [Online]. Available: https://meshtastic.org/docs

[13] Briar Project, "Briar – Peer-to-Peer Encrypted Messaging," briarproject.org, 2024. [Online]. Available: https://briarproject.org

GLOSSARY

Glossary of Technical Terms

Term	Definition
ACK (Acknowledgement)	A control packet sent by the receiver to confirm receipt of a data packet.
Broadcast storm	Uncontrolled exponential proliferation of relay copies of a packet in a mesh network, causing bandwidth saturation.
Composite quality score	A weighted combination of multiple link quality metrics (RSSI, RTT, success rate, retries, disconnects) aggregated into a single 0–100 score.
Connection mode	The operational state of TrailLink: Online, Offline, Reconnecting, or Unstable, determined by backend reachability and hysteresis logic.
DTN (Delay-Tolerant Networking)	A networking paradigm for communication over intermittently connected networks using store-carry-forward delivery [10].
Guidance type	A typed category of the guidance message produced by GuidanceEngine (e.g., signal_improving, move_closer, reconnect).
Hop count	The number of relay hops a packet has already traversed; incremented by each relay node.
Hysteresis	A state transition mechanism that requires consecutive confirmatory observations before changing state, preventing rapid oscillation.
LQE (Link Quality Estimation)	The process of inferring the transmission reliability of a radio link from observable metrics such as RSSI, RTT, and PRR [8].
Nearby Connections	The Google SDK for peer-to-peer communication on Android, abstracting Bluetooth, BLE, and Wi-Fi Direct [2].
P2P_CLUSTER	A Nearby Connections strategy supporting many-to-many device connections within radio range [2].
RSSI (Received Signal Strength Indicator)	A measurement of the power level of a received radio signal in dBm.
TTL (Time To Live)	A forwarding budget decremented by each relay node; when TTL reaches 1, the packet is no longer forwarded.
Trip-scoped	Constrained to the active offline trip, identified by a unique channel code, preventing cross-trip packet delivery.

APPENDICES

Appendix A: Research Instruments or Extended Results

Table A.1 provides extended scoring worked examples covering the full range of quality labels. These examples supplement the three worked examples presented in Section 4.4.3 and demonstrate complete coverage of all five label tiers.

Table A.1: Extended PeerQualityCalculator Scoring Examples

Example	RSSI (dBm)	Status	RTT (ms)	Success	Retries	Disconnects	Score	Label
Ex-1	−52	Connected	180	0.95	0	0	100 (clamped)	Excellent
Ex-2	−68	Connected	450	0.8	1	0	79	Fair
Ex-3	−75	Connected	650	0.7	2	1	58	Fair
Ex-4	— (unavail.)	Connected, seen 20 s	1200	0.7	3	0	68	Good
Ex-5	−83	Connected	1500	0.4	4	2	10	Lost
Ex-6	— (unavail.)	Lost	n/a	0.0	0	1	0 (clamped)	Lost
Ex-7 (boundary)	−70.0	Connected	300	0.9	0	0	90	Excellent

Table A.2 presents the guidance engine test cases used for verification of the eight-branch decision tree, as described in Section 5.4.3.

Table A.2: GuidanceEngine Test Cases

Test Case	Input Condition	Expected Type	Expected Message (summarised)
GE-1	No channel code set	no_peers	Join an offline channel
GE-2	No peers, pending messages > 0	reconnect	Reconnect to forward queued messages
GE-3	No peers, no pending messages	no_peers	Move closer or open Connect Phones
GE-4	Emergency active, best peer = Weak	move_closer	Emergency: move closer to teammate
GE-5	Best peer trend = Improving	signal_improving	Signal improving. Continue in current direction.
GE-6	Best peer trend = Degrading	signal_degrading	Signal degrading. Return to previous position.
GE-7	All peers Weak or Lost	move_closer	Move closer to teammates
GE-8	Best peer = Good, trend = Stable	stay_position	Good connection. Stay in current position.

Appendix B: Academic Integrity and AI Use Disclosure

This appendix records the use of generative AI and other automated tools in preparing this thesis, as required by the module policy of the Sri Lanka Institute of Information Technology (SLIIT) and by the declaration at the front of the thesis. Table A.3 lists each tool, what it was used for, how its output was checked, and which parts of the thesis it affected. The research design, the implementation of the TrailLink component, the testing, the results and the conclusions reported in this thesis are my own work, and I remain responsible for their accuracy, originality and citations.

Table A.4: Disclosure of AI and automated tool use

Tool	**Purpose**	**Verification performed**	**Sections or artefacts affected**
Generative AI assistant (Claude, Anthropic)	Restructuring an earlier draft of this thesis into the SLIIT template and rewording the text.	To be completed by the author before submission.	Text of Chapters 1–9, front matter and appendices.
[Other tool, if any]	[Purpose]	[Verification]	[Sections affected]